

Creating Architecture Diagrams for Smart Plug AI

Here's a comprehensive guide on creating all required diagrams with tool recommendations and step-by-step approaches:

 Tool Selection Recommendation

For your university project, I recommend this **combination**:

Diagram Type	Primary Tool	Backup Tool	Reasoning
Use Case Diagrams	Visual Paradigm	Diagrams.net	Visual Paradigm has excellent UML support with auto-layout
Activity Diagrams	Visual Paradigm	UMLet	Professional UML notation support
Class & ER Diagrams	Visual Paradigm	Diagrams.net	Superior database modeling tools
Sequence Diagrams	Visual Paradigm	UMLet	Best sequence diagram generation
Component Diagrams	Diagrams.net	Visual Paradigm	Great for system architecture visualization
Gantt Charts	Visual Paradigm	Diagrams.net	Built-in project management features

Final Recommendation: Use **Visual Paradigm Community Edition** (free for students) for most diagrams, with [Diagrams.net](#) as backup for component diagrams.

 Suggested File Organization:

Smart_Plug_AI_Diagrams/
| — 1_Use_Case_Diagrams/
| | — 1.1_System_Use_Case.vpp

- | | └─ 1.2_User_Use_Case.vpp
- | | └─ 1.3_Admin_Use_Case.vpp
- | └─ 2_Activity_Diagrams/
 - | | └─ 2.1_Device_Pairing_Flow.vpp
 - | | └─ 2.2_Secure_Command_Flow.vpp
 - | | └─ 2.3_Tamper_Response_Flow.vpp
- | └─ 3_Class_ER_Diagrams/
 - | | └─ 3.1_Class_Diagram_Backend.vpp
 - | | └─ 3.2_Class_Diagram_Firmware.vpp
 - | | └─ 3.3_ER_Diagram_Database.vpp
 - | | └─ 3.4_SQL_DDL_Generated.sql
- | └─ 4_Sequence_Diagrams/
 - | | └─ 4.1_Secure_Control_Sequence.vpp
 - | | └─ 4.2_Tamper_Detection_Sequence.vpp
 - | | └─ 4.3_OTA_Update_Sequence.vpp
- | └─ 5_Component_Diagrams/
 - | | └─ 5.1_System_Architecture.drawio
 - | | └─ 5.2_Deployment_Architecture.drawio
- | └─ 6_Gantt_Charts/
 - | | └─ 6.1_Project_Timeline.vpp
 - | | └─ 6.2_Phase1_Detailed.vpp
- └─ 7_Exports/
 - | └─ All_Diagrams.pdf
 - | └─ Presentation_Slides.pptx

1. HIGH-LEVEL SYSTEM USE CASE DIAGRAM

Primary Actors:

1. **Homeowner** - Primary user, residential customer
2. **Facility Manager** - Commercial/industrial user
3. **System Administrator** - Technical administrator
4. **Security Auditor** - Compliance and security reviewer
5. **IoT Device** - ESP32-S3 smart plug
6. **Mobile Application** - Flutter-based mobile interface
7. **Web Dashboard** - React-based admin interface

System Boundary:

Smart Plug AI Platform - Secure IoT monitoring and control system

Main Use Cases:**Authentication & Security Domain:**

- UC-1: Authenticate with 2FA/MFA
- UC-2: Manage User Profile & Settings
- UC-3: Review Security Audit Logs
- UC-4: Configure Security Policies

Device Management Domain:

- UC-5: Pair New Device (QR + Challenge-Response)
- UC-6: Monitor Real-time Power Usage
- UC-7: Control Device Remotely (Signed Commands)
- UC-8: View Device Status & Health

Data & Analytics Domain:

- UC-9: View Historical Analytics
- UC-10: Generate Usage Reports
- UC-11: Export Data for Analysis

- UC-12: Set Energy Usage Alerts

System Administration Domain:

- UC-13: Manage User Accounts (RBAC)
- UC-14: Configure System Settings
- UC-15: Monitor System Health
- UC-16: Update Firmware (Secure OTA)

Security Operations Domain:

- UC-17: Respond to Tamper Alerts
- UC-18: Review Device Attestation Status
- UC-19: Manage Security Certificates
- UC-20: Investigate Security Incidents

Smart Plug AI: Use Case Diagrams in Mermaid

1. SYSTEM OVERVIEW USE CASE DIAGRAM

graph TB

```
subgraph "Smart Plug AI System"
```

```
%% Actors
```

```
Homeowner[👤 Homeowner]
```

```
Admin[🔑 System Admin]
```

```
Auditor[🔍 Security Auditor]
```

```
Device[⚡ IoT Device]
```

```
MobileApp[📱 Mobile App]
```

```
WebDashboard[🖥️ Web Dashboard]
```

```
%% Use Cases - Authentication & Security
```

```
UC1["UC-1: Authenticate with 2FA/MFA"]
```

```
UC2["UC-2: Manage User Profile & Settings"]
```

```
UC3["UC-3: Review Security Audit Logs"]
```

```
UC4["UC-4: Configure Security Policies"]
```

```
%% Use Cases - Device Management
```

```
UC5["UC-5: Pair New Device<br/>QR + Challenge-Response"]
```

```
UC6["UC-6: Monitor Real-time Power Usage<br/>Encrypted Data"]
```

```
UC7["UC-7: Control Device Remotely<br/>Signed Commands"]
```

```

UC8["UC-8: View Device Status & Health"]

%% Use Cases - Data & Analytics
UC9["UC-9: View Historical Analytics"]
UC10["UC-10: Generate Usage Reports"]
UC11["UC-11: Export Data for Analysis"]
UC12["UC-12: Set Energy Usage Alerts"]

%% Use Cases - System Administration
UC13["UC-13: Manage User Accounts<br/>RBAC"]
UC14["UC-14: Configure System Settings"]
UC15["UC-15: Monitor System Health"]
UC16["UC-16: Update Firmware<br/>Secure OTA"]

%% Use Cases - Security Operations
UC17["UC-17: Respond to Tamper Alerts"]
UC18["UC-18: Review Device Attestation"]
UC19["UC-19: Manage Security Certificates"]
UC20["UC-20: Investigate Security Incidents"]
end

%% Homeowner Relationships
Homeowner --> UC1
Homeowner --> UC2
Homeowner --> UC5
Homeowner --> UC6
Homeowner --> UC7
Homeowner --> UC8
Homeowner --> UC9
Homeowner --> UC10
Homeowner --> UC12
Homeowner -.->|"receives alerts"| UC17

%% Admin Relationships
Admin --> UC1
Admin --> UC3
Admin --> UC4
Admin --> UC13
Admin --> UC14
Admin --> UC15
Admin --> UC16
Admin --> UC18
Admin --> UC19

```

Admin --> UC20

%% Auditor Relationships

Auditor --> UC1

Auditor --> UC3

Auditor --> UC4

Auditor --> UC18

Auditor --> UC19

Auditor --> UC20

%% System Actors

MobileApp --> UC1

MobileApp --> UC5

MobileApp --> UC6

MobileApp --> UC7

MobileApp --> UC8

MobileApp -.->|"push notification"| UC17

WebDashboard --> UC1

WebDashboard --> UC3

WebDashboard --> UC9

WebDashboard --> UC10

WebDashboard --> UC13

WebDashboard --> UC15

WebDashboard --> UC18

WebDashboard --> UC19

WebDashboard --> UC20

Device -.->|"publishes"| UC6

Device -.->|"receives"| UC7

Device -.->|"reports"| UC8

Device -.->|"receives"| UC16

Device -.->|"triggers"| UC17

Device -.->|"provides"| UC18

%% Include Relationships

UC7 -.->|"<<include>>"| UC1

UC13 -.->|"<<include>>"| UC1

UC16 -.->|"<<include>>"| UC1

UC20 -.->|"<<include>>"| UC3

%% Extend Relationships

UC17 -.->|"<<extend>>"| UC12

```
UC18 -.->|"<<extend>>"| UC8
```

```
%% Styling
```

```
classDef actor fill:#bbdefb,stroke:#1565c0
```

```
classDef useCase fill:#e8f5e8,stroke:#2e7d32
```

```
classDef securityUseCase fill:#ffebee,stroke:#c62828
```

```
classDef include fill:#fff3e0,stroke:#ff8f00,dashed
```

```
classDef extend fill:#e1bee7,stroke:#7b1fa2,dashed
```

```
class Homeowner,Admin,Auditor,Device,MobileApp,WebDashboard actor
```

```
class UC1,UC2,UC5,UC6,UC7,UC8,UC9,UC10,UC11,UC12,UC13,UC14,UC15,UC16 useCase
```

```
class UC3,UC4,UC17,UC18,UC19,UC20 securityUseCase
```

```
linkStyle 30 stroke:#ff8f00,stroke-dasharray:5 5
```

```
linkStyle 31 stroke:#ff8f00,stroke-dasharray:5 5
```

```
linkStyle 32 stroke:#ff8f00,stroke-dasharray:5 5
```

```
linkStyle 33 stroke:#7b1fa2,stroke-dasharray:5 5
```

```
linkStyle 34 stroke:#7b1fa2,stroke-dasharray:5 5
```

2. AUTHENTICATION & SECURITY USE CASE DIAGRAM

```
graph TB
```

```
subgraph "Authentication & Security System"
```

```
%% Actors
```

```
User[👤 User]
```

```
Admin[🔑 Admin]
```

```
System[⚡ System]
```

```
%% Authentication Flow
```

```
UC101["UC-101: Login with Credentials"]
```

```
UC102["UC-102: Complete 2FA/MFA"]
```

```
UC103["UC-103: Logout / Terminate Session"]
```

```
UC104["UC-104: Recover Lost Credentials"]
```

```
%% Session Management
```

```
UC105["UC-105: Manage Active Sessions"]
```

```
UC106["UC-106: Renew Expired Tokens"]
```

```
UC107["UC-107: Revoke Compromised Sessions"]
```

```
%% Access Control
```

```
UC108["UC-108: View Role Permissions"]
```

```
UC109["UC-109: Request Elevated Access"]
```

```
UC110["UC-110: Audit Access Attempts"]
```

```
%% Security Features
```

```
    UC111["UC-111: Configure Biometric Auth"]
    UC112["UC-112: Review Login History"]
    UC113["UC-113: Set Password Policies"]
    UC114["UC-114: Manage API Keys"]
```

```
end
```

%% User Relationships

```
User --> UC101
User --> UC102
User --> UC103
User --> UC104
User --> UC105
User --> UC106
User --> UC111
```

%% Admin Relationships

```
Admin --> UC101
Admin --> UC102
Admin --> UC103
Admin --> UC107
Admin --> UC108
Admin --> UC109
Admin --> UC113
Admin --> UC114
```

%% System Relationships

```
System -.->|"validates"| UC101
System -.->|"verifies"| UC102
System -.->|"auto-triggers"| UC106
System -.->|"auto-detects"| UC107
```

%% Include Relationships

```
UC102 -.->|"<<include>>"| UC101
UC109 -.->|"<<include>>"| UC102
UC105 -.->|"<<include>>"| UC101
```

%% Extend Relationships

```
UC104 -.->|"<<extend>>"| UC102
UC107 -.->|"<<extend>>"| UC110
```

%% Notes

```
subgraph "2FA/MFA Options"
    TOTP["TOTP (Google Authenticator)"]
```



```

    SMS["SMS Verification"]
    Push["Push Notification"]
    Bio["Biometric Verification"]
end

subgraph "RBAC Roles"
    Role1["Admin: Full system access"]
    Role2["User: Device management"]
    Role3["Viewer: Read-only"]
    Role4["Auditor: Security logs only"]
    Role5["Service: API access"]
end

TOTP --> UC102
SMS --> UC102
Push --> UC102
Bio --> UC102

Role1 --> UC108
Role2 --> UC108
Role3 --> UC108
Role4 --> UC108
Role5 --> UC108

%% Styling
classDef actor fill:#bbdefb,stroke:#1565c0
classDef authUseCase fill:#e8f5e8,stroke:#2e7d32
classDef securityUseCase fill:#ffebee,stroke:#c62828
classDef option fill:#fff3e0,stroke:#ff8f00
classDef role fill:#e1bee7,stroke:#7b1fa2
classDef include fill:#ff8f00,stroke-dasharray:5 5
classDef extend fill:#7b1fa2,stroke-dasharray:5 5

class User,Admin,System actor
class UC101,UC102,UC103,UC104,UC105,UC106 authUseCase
class UC107,UC108,UC109,UC110,UC111,UC112,UC113,UC114 securityUseCase
class TOTP,SMS,Push,Bio option
class Role1,Role2,Role3,Role4,Role5 role

linkStyle 12 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 13 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 14 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 15 stroke:#7b1fa2,stroke-dasharray:5 5

```

```
linkStyle 16 stroke:#7b1fa2,stroke-dasharray:5 5
```

3. DEVICE MANAGEMENT USE CASE DIAGRAM

```
graph TB
    subgraph "Device Management System"
        %% Actors
        User[👤 User]
        Admin[🔑 Admin]
        Mobile[📱 Mobile App]
        Device[⚡ IoT Device]
        Backend[☁️ Cloud Backend]

        %% Device Provisioning
        UC201["UC-201: Pair New Device<br/>QR Scan"]
        UC202["UC-202: Verify Device Identity"]
        UC203["UC-203: Complete Challenge-Response"]
        UC204["UC-204: Store Device Certificate"]

        %% Device Control
        UC205["UC-205: Turn Device On/Off"]
        UC206["UC-206: Schedule Device Operations"]
        UC207["UC-207: Toggle Device State"]
        UC208["UC-208: Send Signed Command"]

        %% Device Monitoring
        UC209["UC-209: View Real-time Telemetry"]
        UC210["UC-210: Monitor Device Health"]
        UC211["UC-211: Check Power Consumption"]
        UC212["UC-212: Receive Device Alerts"]

        %% Device Configuration
        UC213["UC-213: Rename/Label Device"]
        UC214["UC-214: Set Device Location"]
        UC215["UC-215: Configure Alert Thresholds"]
        UC216["UC-216: Update Device Settings"]

        %% Device Security
        UC217["UC-217: Verify Device Attestation"]
        UC218["UC-218: Review Tamper Status"]
        UC219["UC-219: Revoke Device Access"]
        UC220["UC-220: Perform Security Audit"]
    end
end
```

%% User Relationships

User --> UC201
User --> UC205
User --> UC206
User --> UC209
User --> UC211
User --> UC212
User --> UC213
User --> UC214
User --> UC215

%% Admin Relationships

Admin --> UC201
Admin --> UC202
Admin --> UC203
Admin --> UC204
Admin --> UC217
Admin --> UC218
Admin --> UC219
Admin --> UC220

%% Mobile App Relationships

Mobile -->|"scanner"| UC201
Mobile -->|"interface"| UC205
Mobile -->|"display"| UC209
Mobile -->|"notifications"| UC212

%% IoT Device Relationships

Device -->|"provides device ID"| UC202
Device -->|"responds to challenge"| UC203
Device -->|"receives commands"| UC208
Device -->|"publishes telemetry"| UC209
Device -->|"triggers alerts"| UC212
Device -->|"provides attestation"| UC217
Device -->|"detects tampering"| UC218

%% Backend Relationships

Backend -->|"validates"| UC202
Backend -->|"generates challenge"| UC203
Backend -->|"stores certificate"| UC204
Backend -->|"signs commands"| UC208
Backend -->|"verifies attestation"| UC217

%% Include Relationships

```
UC201 -.->|"<<include>>"| UC202
UC201 -.->|"<<include>>"| UC203
UC201 -.->|"<<include>>"| UC204
UC208 -.->|"<<include>>"| UC217
```

%% Extend Relationships

```
UC212 -.->|"<<extend>>"| UC209
UC220 -.->|"<<extend>>"| UC217
UC220 -.->|"<<extend>>"| UC218
```

%% Security Processes

```
subgraph "Secure Pairing Flow"
    Step1["1. Device shows QR with ID + challenge"]
    Step2["2. Mobile app scans QR"]
    Step3["3. App sends to backend for verification"]
    Step4["4. Backend signs challenge response"]
    Step5["5. Device verifies with ATECC608A"]
    Step6["6. Certificate exchange completes"]
end
```

```
subgraph "Signed Command Process"
    Cmd1["1. User initiates command in app"]
    Cmd2["2. Backend signs with ECDSA private key"]
    Cmd3["3. Command sent via MQTT over TLS 1.3"]
    Cmd4["4. Device verifies with server public key"]
    Cmd5["5. ATECC608A validates signature"]
    Cmd6["6. Command executed if valid"]
end
```

```
Step1 -.-> UC201
Step6 -.-> UC204
Cmd1 -.-> UC205
Cmd6 -.-> UC208
```

%% Styling

```
classDef actor fill:#bbdefb,stroke:#1565c0
classDef provisioning fill:#e8f5e8,stroke:#2e7d32
classDef control fill:#bbdefb,stroke:#1565c0
classDef monitoring fill:#fff3e0,stroke:#ff8f00
classDef security fill:#ffebee,stroke:#c62828
classDef process fill:#f3e5f5,stroke:#7b1fa2
classDef include fill:#ff8f00,stroke-dasharray:5 5
```

```

classDef extend fill:#7b1fa2,stroke-dasharray:5 5

class User,Admin,Mobile,Device,Backend actor
class UC201,UC202,UC203,UC204 provisioning
class UC205,UC206,UC207,UC208 control
class UC209,UC210,UC211,UC212,UC213,UC214,UC215,UC216 monitoring
class UC217,UC218,UC219,UC220 security
class Step1,Step2,Step3,Step4,Step5,Step6,Cmd1,Cmd2,Cmd3,Cmd4,Cmd5,Cmd6
process

```

```

linkStyle 30 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 31 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 32 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 33 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 34 stroke:#7b1fa2,stroke-dasharray:5 5
linkStyle 35 stroke:#7b1fa2,stroke-dasharray:5 5
linkStyle 36 stroke:#7b1fa2,stroke-dasharray:5 5

```

4. DATA ANALYTICS USE CASE DIAGRAM

```

graph TB
    subgraph "Analytics & Reporting System"
        %% Actors
        Homeowner[👤 Homeowner]
        FacilityManager[🏢 Facility Manager]
        EnergyAnalyst[📊 Energy Analyst]
        Dashboard[🖥️ Web Dashboard]
        Mobile[📱 Mobile App]
        Database[🗄️ Database System]

        %% Real-time Monitoring
        UC301["UC-301: View Live Power Stream"]
        UC302["UC-302: Monitor Current Consumption"]
        UC303["UC-303: Track Voltage Stability"]
        UC304["UC-304: Watch Energy Usage Patterns"]

        %% Historical Analysis
        UC305["UC-305: Analyze Daily Usage"]
        UC306["UC-306: Review Weekly Trends"]
        UC307["UC-307: Compare Monthly Data"]
        UC308["UC-308: View Year-over-Year"]

        %% Cost & Efficiency
        UC309["UC-309: Calculate Energy Costs"]
    end

```

```
UC310["UC-310: Estimate Bill Projections"]
UC311["UC-311: Identify Waste Patterns"]
UC312["UC-312: Suggest Efficiency Improvements"]
```

%% Reporting

```
UC313["UC-313: Generate Usage Reports"]
UC314["UC-314: Export Data (CSV/PDF)"]
UC315["UC-315: Schedule Automated Reports"]
UC316["UC-316: Share Reports Securely"]
```

%% Alerting

```
UC317["UC-317: Set Usage Thresholds"]
UC318["UC-318: Configure Alert Rules"]
UC319["UC-319: Receive Usage Alerts"]
UC320["UC-320: Review Alert History"]
```

%% Advanced Analytics

```
UC321["UC-321: Run Predictive Analysis"]
UC322["UC-322: Generate Load Profiles"]
UC323["UC-323: Perform Anomaly Detection"]
UC324["UC-324: Create Custom Dashboards"]
```

end

%% Homeowner Relationships

```
Homeowner --> UC301
Homeowner --> UC302
Homeowner --> UC305
Homeowner --> UC309
Homeowner --> UC310
Homeowner --> UC311
Homeowner --> UC317
Homeowner --> UC319
```

%% Facility Manager Relationships

```
FacilityManager --> UC301
FacilityManager --> UC306
FacilityManager --> UC307
FacilityManager --> UC309
FacilityManager --> UC313
FacilityManager --> UC314
FacilityManager --> UC315
FacilityManager --> UC321
FacilityManager --> UC322
```

FacilityManager --> UC324

%% Energy Analyst Relationships

EnergyAnalyst --> UC307

EnergyAnalyst --> UC308

EnergyAnalyst --> UC321

EnergyAnalyst --> UC322

EnergyAnalyst --> UC323

EnergyAnalyst --> UC324

%% Web Dashboard Relationships

Dashboard -->|"displays"| UC301

Dashboard -->|"charts"| UC305

Dashboard -->|"generates"| UC313

Dashboard -->|"exports"| UC314

Dashboard -->|"customizes"| UC324

%% Mobile App Relationships

Mobile -->|"streams"| UC301

Mobile -->|"shows"| UC302

Mobile -->|"notifies"| UC319

%% Database Relationships

Database -->|"stores historical data"| UC305

Database -->|"queries for reports"| UC313

Database -->|"provides training data"| UC321

%% Include Relationships

UC301 -->|"<<include>>"| UC302

UC305 -->|"<<include>>"| UC304

UC313 -->|"<<include>>"| UC309

UC321 -->|"<<include>>"| UC305

%% Extend Relationships

UC319 -->|"<<extend>>"| UC301

UC323 -->|"<<extend>>"| UC304

UC324 -->|"<<extend>>"| UC301

%% Data Flow Diagram

subgraph "Real-time Data Flow"

Flow1["1. Device → MQTT over TLS 1.3"]

Flow2["2. Backend → WebSocket over TLS"]

Flow3["3. Client → Encrypted display"]

```

    Flow4["4. Database → Encrypted at rest"]
    Flow5["5. Analytics → Field-level encryption"]
end

subgraph "Anomaly Detection"
    Anomaly1["• Unusual power spikes"]
    Anomaly2["• Unexpected consumption patterns"]
    Anomaly3["• Tamper detection via MAX6316"]
    Anomaly4["• Firmware integrity violations"]
    Anomaly5["• Network security anomalies"]
end

Flow1 -.-> UC301
Flow5 -.-> UC323
Anomaly3 -.-> UC323

%% Styling
classDef actor fill:#bbdefb,stroke:#1565c0
classDef realtime fill:#e8f5e8,stroke:#2e7d32
classDef historical fill:#bbdefb,stroke:#1565c0
classDef analytics fill:#fff3e0,stroke:#ff8f00
classDef reporting fill:#e1bee7,stroke:#7b1fa2
classDef advanced fill:#ffebee,stroke:#c62828
classDef flow fill:#c8e6c9,stroke:#2e7d32
classDef include fill:#ff8f00,stroke-dasharray:5 5
classDef extend fill:#7b1fa2,stroke-dasharray:5 5

class Homeowner,FacilityManager,EnergyAnalyst,Dashboard,Mobile,Database actor
class UC301,UC302,UC303,UC304 realtime
class UC305,UC306,UC307,UC308 historical
class UC309,UC310,UC311,UC312,UC317,UC318,UC319,UC320 analytics
class UC313,UC314,UC315,UC316 reporting
class UC321,UC322,UC323,UC324 advanced
class

Flow1,Flow2,Flow3,Flow4,Flow5,Anomaly1,Anomaly2,Anomaly3,Anomaly4,Anomaly5 flow

linkStyle 34 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 35 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 36 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 37 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 38 stroke:#7b1fa2,stroke-dasharray:5 5
linkStyle 39 stroke:#7b1fa2,stroke-dasharray:5 5
linkStyle 40 stroke:#7b1fa2,stroke-dasharray:5 5

```


5. SECURITY OPERATIONS USE CASE DIAGRAM

```
graph TB
    subgraph "Security Operations Center"
        %% Actors
        SecOfficer[👤 Security Officer]
        SysAdmin[🔑 System Admin]
        Auditor[📋 Compliance Auditor]
        Device[⚡ IoT Device]
        AlertSystem[📢 Alert System]
        SIEM[📊 SIEM System]

        %% Threat Detection
        UC401["UC-401: Detect Physical Tampering"]
        UC402["UC-402: Identify Anomalous Activity"]
        UC403["UC-403: Monitor Network Intrusions"]
        UC404["UC-404: Track Authentication Failures"]

        %% Incident Response
        UC405["UC-405: Receive Security Alerts"]
        UC406["UC-406: Triage Security Incidents"]
        UC407["UC-407: Contain Security Breaches"]
        UC408["UC-408: Remediate Security Issues"]

        %% Forensics & Investigation
        UC409["UC-409: Collect Forensic Evidence"]
        UC410["UC-410: Analyze Attack Patterns"]
        UC411["UC-411: Trace Compromised Devices"]
        UC412["UC-412: Document Security Incidents"]

        %% Compliance & Auditing
        UC413["UC-413: Generate Compliance Reports"]
        UC414["UC-414: Audit Security Controls"]
        UC415["UC-415: Verify Certificate Validity"]
        UC416["UC-416: Review Access Logs"]

        %% Device Security
        UC417["UC-417: Verify Device Attestation"]
        UC418["UC-418: Manage Security Certificates"]
        UC419["UC-419: Revoke Compromised Devices"]
        UC420["UC-420: Enforce Security Policies"]
    end
```

```
%% Security Management
UC421["UC-421: Update Security Configurations"]
UC422["UC-422: Manage Encryption Keys"]
UC423["UC-423: Conduct Security Drills"]
UC424["UC-424: Train Security Personnel"]
end
```

%% Security Officer Relationships

```
SecOfficer --> UC401
SecOfficer --> UC405
SecOfficer --> UC406
SecOfficer --> UC407
SecOfficer --> UC408
SecOfficer --> UC409
SecOfficer --> UC410
SecOfficer --> UC411
SecOfficer --> UC412
SecOfficer --> UC421
SecOfficer --> UC422
SecOfficer --> UC423
SecOfficer --> UC424
```

%% System Admin Relationships

```
SysAdmin --> UC401
SysAdmin --> UC407
SysAdmin --> UC408
SysAdmin --> UC415
SysAdmin --> UC417
SysAdmin --> UC418
SysAdmin --> UC419
SysAdmin --> UC420
```

%% Compliance Auditor Relationships

```
Auditor --> UC413
Auditor --> UC414
Auditor --> UC416
Auditor --> UC417
```

%% IoT Device Relationships

```
Device -->|"triggers via MAX6316"| UC401
Device -->|"provides attestation data"| UC417
Device -->|"uses certificates"| UC418
Device -->|"gets revoked"| UC419
```

%% Alert System Relationships

```
AlertSystem -.->|"sends real-time alerts"| UC405
AlertSystem -.->|"escalates tamper events"| UC401
AlertSystem -.->|"notifies anomalies"| UC402
AlertSystem -.->|"reports failed logins"| UC404
```

%% SIEM System Relationships

```
SIEM -.->|"correlates events"| UC402
SIEM -.->|"analyzes traffic"| UC403
SIEM -.->|"provides logs"| UC410
SIEM -.->|"aggregates audit data"| UC416
```

%% Include Relationships

```
UC405 -.->|"<<include>>"| UC401
UC405 -.->|"<<include>>"| UC402
UC405 -.->|"<<include>>"| UC403
UC405 -.->|"<<include>>"| UC404
```

```
UC407 -.->|"<<include>>"| UC406
UC408 -.->|"<<include>>"| UC407
```

```
UC414 -.->|"<<include>>"| UC416
UC414 -.->|"<<include>>"| UC417
```

%% Extend Relationships

```
UC419 -.->|"<<extend>>"| UC401
UC419 -.->|"<<extend>>"| UC403
UC419 -.->|"<<extend>>"| UC411
```

```
UC423 -.->|"<<extend>>"| UC405
UC423 -.->|"<<extend>>"| UC407
```

%% Security Frameworks

```
subgraph "Security Framework Compliance"
    NIST["NIST Cybersecurity Framework"]
    OWASP["OWASP IoT Security Top 10"]
    SABS["SABS/ICASA Compliance"]
    SOC2["SOC2 Type II Controls"]
    POPIA["POPIA/GDPR Data Protection"]
    Crypto["Certificate: ECDSA P256, TLS 1.3"]
end
```

```
NIST -.-> UC414
OWASP -.-> UC414
SABS -.-> UC413
SOC2 -.-> UC413
POPIA -.-> UC413
Crypto -.-> UC418
```

%% Tamper Detection Flow

```
subgraph "Tamper Detection Flow"
    T1["1. MAX6316 detects enclosure opening"]
    T2["2. Triggers ESP32 reset"]
    T3["3. Sends tamper alert via MQTT/TLS"]
    T4["4. Backend logs security event"]
    T5["5. User notified via push notification"]
    T6["6. Device marked as 'tampered'"]
    T7["7. Requires re-verification"]
end
```

```
T1 -.-> UC401
T7 -.-> UC417
```

%% Device Attestation

```
subgraph "Device Attestation"
    A1["• Firmware hash verification"]
    A2["• Secure boot status check"]
    A3["• Flash encryption status"]
    A4["• ATECC608A key validation"]
    A5["• Certificate chain verification"]
    A6["• Reported every 24h"]
end
```

```
A1 -.-> UC417
A6 -.-> UC417
```

%% Styling

```
classDef actor fill:#bbdefb,stroke:#1565c0
classDef detection fill:#ffebee,stroke:#c62828
classDef response fill:#fff3e0,stroke:#ff8f00
classDef forensics fill:#e1bee7,stroke:#7b1fa2
classDef compliance fill:#c8e6c9,stroke:#2e7d32
classDef framework fill:#b2dfdb,stroke:#00695c
classDef flow fill:#f3e5f5,stroke:#7b1fa2
```

```

classDef include fill:#ff8f00,stroke-dasharray:5 5
classDef extend fill:#7b1fa2,stroke-dasharray:5 5

class SecOfficer,SysAdmin,Auditor,Device,AlertSystem,SIEM actor
class UC401,UC402,UC403,UC404 detection
class UC405,UC406,UC407,UC408 response
class UC409,UC410,UC411,UC412 forensics
class UC413,UC414,UC415,UC416,UC417,UC418,UC419,UC420 compliance
class NIST,OWASP,SABS,SOC2,POPIA,Crypto framework
class T1,T2,T3,T4,T5,T6,T7,A1,A2,A3,A4,A5,A6 flow

```

```

linkStyle 36 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 37 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 38 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 39 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 40 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 41 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 42 stroke:#ff8f00,stroke-dasharray:5 5
linkStyle 43 stroke:#7b1fa2,stroke-dasharray:5 5
linkStyle 44 stroke:#7b1fa2,stroke-dasharray:5 5
linkStyle 45 stroke:#7b1fa2,stroke-dasharray:5 5
linkStyle 46 stroke:#7b1fa2,stroke-dasharray:5 5

```

By User Role Access:

```

pie title Use Case Distribution by Role
    "Homeowner" : 45
    "System Admin" : 25
    "Security Auditor" : 15
    "System Components" : 10
    "Facility Manager" : 5

```

By Implementation Priority:

```

graph LR
    P1["Phase 1 (Weeks 1-12)<br/>Authentication & Device Control"] -->
    P2["Phase 2 (Months 5-8)<br/>Analytics & Compliance"] -->
    P3["Phase 3 (Months 9-12)<br/>Enterprise & Advanced Security"]

    P1_UC["UC-1, UC-5, UC-7, UC-17"]
    P2_UC["UC-9, UC-13, UC-18, UC-413"]
    P3_UC["UC-19, UC-20, UC-321, UC-419"]

    P1 --> P1_UC
    P2 --> P2_UC
    P3 --> P3_UC

```

Smart Plug AI: Activity Diagrams in Mermaid

1. SECURE DEVICE PAIRING ACTIVITY DIAGRAM

flowchart TD

```
A[Start: Device Unpaired] --> B[ESP32-S3 Generates<br>Device ID & Private Key]
B --> C[Store Private Key<br>in ATECC608A Slot 0]
C --> D[Generate QR Code with:<br>• Device ID<br>• Public Key<br>• Challenge Token]
```

```
subgraph DeviceActivity [Device Actions]
    D --> E[Display QR Code<br>on LED/OLED]
    E --> F{Wait for Pairing Response}
end
```

```
subgraph UserActivity [User Actions]
    G[User Opens Mobile App] --> H[Scan QR Code with<br>Camera]
    H --> I[Extract Device ID,<br>Public Key & Challenge]
end
```

```
F --> J{Received Response?}
```

```
subgraph BackendActivity [Backend Server Actions]
    K[Receive Pairing Request] --> L[Validate Challenge<br>Token]
    L --> M[Generate Response<br>Signed with Server Key]
    M --> N[Store Device Association<br>in Database]
    N --> O[Return Signed Response<br>to Mobile App]
end
```

```
subgraph SecurityVerification [Security Verification]
    P[Device Receives<br>Signed Response] --> Q[ATECC608A Verifies<br>Server Signature]
    Q --> R{Signature Valid?}
    R -->|Yes| S[Store Server Public Key<br>in ATECC608A Slot 1]
    R -->|No| T[Log Failed Attempt<br>to EEPROM]
end
```

```
S --> U[Send Device Attestation:<br>• Firmware Hash<br>• Secure Boot Status<br>• Flash Encryption Status]
```

```
subgraph Finalization [Pairing Completion]
    U --> V[Backend Verifies<br>Attestation]
end
```

```

V --> W[Issue Device Certificate<br>signed by Device CA]
W --> X[Secure OTA of Certificate<br>to Device]
X --> Y[Device Stores Certificate<br>in ATECC608A]
Y --> Z[Pairing Complete!<br>Device Ready for Use]

```

end

```

T --> AA[Failure: Display Error<br>LED Pattern]

```

```

AA --> AB[Reset to Pairing Mode<br>after 30s]

```

%% Connections

```

I --> K

```

```

O --> P

```

%% Styling

```

classDef device fill:#e3f2fd,stroke:#1976d2,stroke-width:2px

```

```

classDef user fill:#e8f5e8,stroke:#2e7d32,stroke-width:2px

```

```

classDef backend fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px

```

```

classDef security fill:#ffebee,stroke:#c62828,stroke-width:2px

```

```

classDef success fill:#e8f5e8,stroke:#2e7d32,stroke-width:3px

```

```

classDef failure fill:#ffebee,stroke:#c62828,stroke-width:3px

```

```

class B,C,D,E,F,J,P,Q,R,S,T,U,X,Y,AB device

```

```

class G,H,I user

```

```

class K,L,M,N,O,V,W backend

```

```

class Q,R security

```

```

class Z success

```

```

class AA,AB failure

```

%% Notes

```

note1> **Security Notes: **<br>• ATECC608A prevents key extraction<br>• ECDSA
P256 signatures<br>• Challenge-response prevents replay<br>• Device attestation
ensures integrity]

```

2. SECURE COMMAND EXECUTION ACTIVITY DIAGRAM

flowchart TD

```

A[User Taps Control Button<br>in Mobile App] --> B[App Sends Request<br>to Backend API]

```

```

B --> C[Backend Validates<br>User JWT Token]

```

```

C --> D{RBAC Check Passed?}

```

```

D -->|Yes| E[Generate Command Payload:<br>• Command Type<br>• Timestamp<br>• Nonce]

```

```

D -->|No| F[Return 403 Forbidden]

```

```

F --> G[App Shows Error:<br>Insufficient Permissions]

```

```

E --> H[ECDSA Sign Command<br>with Server Private Key]
H --> I[Publish to MQTT Broker<br>over TLS 1.3: Port 8883]

subgraph MQTTTransmission [MQTT Transmission]
    I --> J[TLS 1.3 Handshake with<br>Client Certificate Auth]
    J --> K[Device Certificate<br>Validation]
    K --> L[Encrypted Delivery<br>to Specific Topic]
end

subgraph DeviceVerification [Device Verification Process]
    M[Device Receives Message] --> N[Extract Signature<br>from Payload]
    N --> O[Compute SHA-256 Hash<br>of Command Data]
    O --> P[ATECC608A Verifies Signature<br>using Server Public Key Slot 1]
    P --> Q{Signature Valid?}
    Q -->|Yes| R[Check Timestamp Freshness<br>±5 minutes]
    Q -->|No| S[Log Security Event:<br>Invalid Signature]
end

R --> T{Timestamp Valid?}
T -->|Yes| U[Execute Command:<br>Relay ON/OFF/Toggle]
T -->|No| V[Log Security Event:<br>Replay Attempt]

subgraph CommandExecution [Command Execution]
    U --> W[Update Relay State]
    W --> X[Store State in<br>Encrypted EEPROM]
    X --> Y[Publish Status Update<br>via MQTT]
end

subgraph BackendUpdate [Backend Updates]
    Y --> Z[Backend Receives<br>Status Update]
    Z --> AA[Update Database:<br>Device State]
    AA --> AB[Push Update via<br>WebSocket over TLS]
    AB --> AC[App Updates UI<br>in Real-time]
end

V --> AD[Discard Command<br>Silently]
S --> AD

AD --> AE[Increment Security<br>Failure Counter]
AE --> AF{Counter > Threshold?}
AF -->|Yes| AG[Mark Device as<br>Potentially Compromised]
AF -->|No| AH[Continue Normal<br>Operations]

```



```
AG --> AI[Alert Security Team<br>via Dashboard]
```

```
AI --> AJ[Require Manual<br>Security Review]
```

```
%% Success Path Styling
```

```
U -->|Success| AK[Command Executed<br>Successfully]
```

```
%% Styling
```

```
classDef userAction fill:#bbdefb,stroke:#1565c0
classDef backendAction fill:#e1bee7,stroke:#7b1fa2
classDef securityCheck fill:#ffcdd2,stroke:#c62828,stroke-width:2px
classDef deviceAction fill:#c8e6c9,stroke:#2e7d32
classDef mqtt fill:#fff3e0,stroke:#ef6c00
classDef success fill:#e8f5e8,stroke:#2e7d32,stroke-width:3px
classDef failure fill:#ffebee,stroke:#c62828,stroke-width:3px
classDef alert fill:#ffecb3,stroke:#ff8f00
```

```
class A,B,G userAction
class C,D,E,F,H,I,Z,AA,AB backendAction
class D,P,Q,R,T securityCheck
class M,N,O,U,W,X,Y deviceAction
class J,K,L mqtt
class AK success
class F,S,V,AD,AE failure
class AG,AJ alert
```

```
%% Notes
```

```
note2> **Security Features:**<br>• ECDSA P256 signatures<br>• Nonce prevents replay<br>• ATECC608A hardware verification<br>• TLS 1.3 encryption<br>• Rate limiting per device<br>• Comprehensive audit logging]
```

3. TAMPER DETECTION & RESPONSE ACTIVITY DIAGRAM

```
flowchart TD
```

```
A[Physical Tamper Detected] --> B[MAX6316 Watchdog<br>Triggers Immediately]
```

```
subgraph ImmediateResponse [Immediate Hardware Response]
```

```
    B --> C[ESP32-S3 Hard Reset<br>within 100ms]
```

```
    C --> D[Tamper LED Indicator<br>Solid Red]
```

```
    D --> E[Disable Relay Control<br>Safety Mode]
```

```
end
```

```
subgraph TamperLogging [Tamper Event Logging]
```

```
    E --> F[Write Tamper Event to<br>24C256 Encrypted EEPROM]
```

```

    F --> G[Increment Tamper Counter<br>in Secure Storage]
    G --> H[Store Timestamp,<br>Event Details]
end

subgraph SecureBootCheck [Secure Boot Verification]
    H --> I[ESP32-S3 Secure Boot V2<br>Verifies Firmware]
    I --> J{Secure Boot Valid?}
    J -->|Yes| K[Continue to<br>Network Alert]
    J -->|No| L[Enter Lockdown Mode:<br>Only Serial Debug]
end

subgraph NetworkAlert [Network Alerting]
    K --> M{Network Available?}
    M -->|Yes| N[Send Tamper Alert via<br>MQTT over TLS 1.3]
    M -->|No| O[Store Alert in Queue<br>for Later Transmission]

    N --> P[Backend Receives Alert]
    P --> Q[Log Security Event<br>in PostgreSQL]
    Q --> R[Mark Device as<br>'TAMPERED' in Database]
    R --> S[Send Push Notification<br>to User Mobile App]
    R --> T[Send Email Alert<br>to Registered User]
end

subgraph UserResponse [User Response Required]
    S --> U[User Opens App<br>Sees Tamper Alert]
    U --> V[User Must Verify<br>Physical Device]
    V --> W[User Initiates<br>Device Re-verification]

    W --> X[Challenge-Response<br>Authentication]
    X --> Y{Authentication<br>Successful?}
    Y -->|Yes| Z[Clear Tamper Flag<br>in Backend]
    Y -->|No| AA[Device Remains<br>Quarantined]
end

Z --> AB[Send Clear Command<br>via Secure OTA]
AB --> AC[Device Clears<br>Tamper Status]
AC --> AD[Normal Operations<br>Resumed]

subgraph SecureErase [Optional Secure Erase]
    L --> AE[Erase Wi-Fi Credentials<br>from NVS]
    AE --> AF[Erase Session Tokens<br>& Temporary Keys]
    AF --> AG[Clear Telemetry Cache]
    AG --> AH[Device Requires<br>Factory Reset]
end

```

end

```
O --> AI[Retry Transmission<br>Every 5 Minutes]
AI --> AJ{Max Retries<br>Exceeded?}
AJ -->|Yes| AK[Enter Deep Sleep<br>Conserve Power]
AJ -->|No| M
```

```
AA --> AL[Security Team<br>Manual Investigation]
AL --> AM[Physical Inspection<br>Required]
AM --> AN{Device Compromised?}
AN -->|No| Z
AN -->|Yes| AO[Revoke Device<br>Certificate]
AO --> AP[Add to Certificate<br>Revocation List]
AP --> AQ[Device Permanently<br>Disabled]
```

%% Styling

```
classDef hardware fill:#ffcdd2,stroke:#c62828,stroke-width:2px
classDef security fill:#fff3e0,stroke:#ff8f00
classDef network fill:#bbdefb,stroke:#1565c0
classDef user fill:#c8e6c9,stroke:#2e7d32
classDef recovery fill:#e8f5e8,stroke:#2e7d32,stroke-width:2px
classDef failure fill:#f5f5f5,stroke:#757575
classDef critical fill:#ffebee,stroke:#c62828,stroke-width:3px
```

```
class A,B,C,D,E hardware
class F,G,H,I,J security
class M,N,O,P,Q,R,S,T,AI,AJ network
class U,V,W,X,Y user
class Z,AB,AC,AD recovery
class L,AE,AF,AG,AH,AO,AP,AQ failure
class AK critical
```

%% Critical Path Indicators

```
linkStyle 0 stroke:#c62828,stroke-width:3px,color:#c62828
linkStyle 1 stroke:#c62828,stroke-width:3px,color:#c62828
linkStyle 2 stroke:#c62828,stroke-width:3px,color:#c62828
```

%% Notes

```
note3> **Tamper Response Levels:**<br>1. Immediate: Hardware reset (100ms)<br>2. Short-term: Network alerts<br>3. Long-term: Re-verification required<br>4. Permanent: Certificate revocation]
```

4. SECURE OTA FIRMWARE UPDATE ACTIVITY DIAGRAM

flowchart TD

A[Start OTA Update Process] --> B[Backend Checks for
Available Updates]

subgraph UpdateAvailability [Update Check]

B --> C{New Firmware Available?}

C -->|Yes| D[Verify Device Compatibility
& Update Policy]

C -->|No| E[Log: No Updates
Available]

E --> F[End Process]

end

D --> G[Generate Signed Update
Manifest with RSA-3072]

subgraph ManifestVerification [Manifest Verification]

G --> H[Send Manifest to Device
via MQTT over TLS 1.3]

H --> I[Device Receives Manifest]

I --> J[ESP32-S3 Verifies
Manifest Signature]

J --> K{Signature Valid?}

K -->|Yes| L[Check Firmware Version
against Rollback Protection]

K -->|No| M[Log Security Event:
Invalid Manifest]

M --> N[Abort Update
Alert Backend]

end

subgraph FirmwareDownload [Firmware Download]

L --> O[Download Firmware from
HTTPS Endpoint with TLS 1.3]

O --> P[Verify TLS Certificate
Pinning]

P --> Q[Stream Firmware to
ESP32 Flash Memory]

Q --> R[Compute SHA-256 Hash
of Downloaded Firmware]

end

subgraph IntegrityVerification [Integrity Verification]

R --> S[Compare Hash with
Manifest Signature]

S --> T{Hashes Match?}

T -->|Yes| U[Prepare to Flash
to OTA Partition]

T -->|No| V[Delete Corrupted
Firmware]

V --> W[Report Integrity
Failure to Backend]

end

subgraph FirmwareFlash [Firmware Flashing]

U --> X[Switch Boot Partition
to OTA Section]

X --> Y[Set Boot Flag for
New Firmware]

Y --> Z[Trigger ESP32 Restart
for Firmware Activation]

end

```

subgraph PostUpdate [Post-Update Verification]
    Z --> AA[ESP32 Boots from<br>New Firmware]
    AA --> AB[Secure Boot V2<br>Verifies Signature]
    AB --> AC{Secure Boot<br>Successful?}
    AC -->|Yes| AD[Initialize New<br>Firmware]
    AC -->|No| AE[Auto-Rollback to<br>Previous Firmware]

    AD --> AF[Generate Device<br>Attestation Report]
    AF --> AG[Send Attestation to<br>Backend for Verification]
    AG --> AH{Attestation<br>Valid?}
    AH -->|Yes| AI[Update Successful!<br>Clear Update Flag]
    AH -->|No| AJ[Mark Device as<br>Potentially Compromised]
end

AE --> AK[Boot Previous<br>Firmware]
AK --> AL[Report Rollback<br>to Backend]
AL --> AM[Schedule Manual<br>Update Investigation]

AJ --> AN[Security Team Alert:<br>Invalid Attestation]
AN --> AO[Device Quarantine<br>until Investigation]

%% Rollback Protection
L --> AP{Version >= Current?}
AP -->|Yes| O
AP -->|No| AQ[Reject Update:<br>Rollback Prevention]
AQ --> AR[Log Security Event<br>Rollback Attempt]

%% Styling
classDef backend fill:#e1bee7,stroke:#7b1fa2
classDef security fill:#ffcdd2,stroke:#c62828,stroke-width:2px
classDef device fill:#c8e6c9,stroke:#2e7d32
classDef network fill:#bbdefb,stroke:#1565c0
classDef success fill:#e8f5e8,stroke:#2e7d32,stroke-width:3px
classDef failure fill:#ffebee,stroke:#c62828,stroke-width:2px
classDef warning fill:#fff3e0,stroke:#ff8f00

class B,D,G,H backend
class J,K,L,P,S,T,AB,AC,AP security
class I,O,Q,R,U,X,Y,Z,AA,AD,AF device
class O,P network
class AI,F success
class M,N,V,W,AE,AJ,AQ,AR failure
class AL,AM,AO warning

```

```
%% Critical Verification Points
```

```
style K fill:#ffebee,stroke:#c62828,stroke-width:3px
style T fill:#ffebee,stroke:#c62828,stroke-width:3px
style AC fill:#ffebee,stroke:#c62828,stroke-width:3px
style AH fill:#ffebee,stroke:#c62828,stroke-width:3px
```

```
%% Notes
```

```
note4> **OTA Security Features:**<br>• RSA-3072 signed manifests<br>• TLS 1.3
for all transfers<br>• Certificate pinning<br>• SHA-256 integrity
verification<br>• Secure boot verification<br>• Rollback protection<br>• Device
attestation post-update]
```

5. REAL-TIME TELEMETRY DATA FLOW ACTIVITY DIAGRAM

```
flowchart TD
```

```
A[SCT-013 Current Sensor] --> B[Measure AC Current<br>via 33Ω Burden
Resistor]
```

```
subgraph SensorReading [Sensor Data Acquisition]
```

```
  B --> C[ESP32 ADC Samples<br>at 1kHz Frequency]
  C --> D[Digital Filtering &<br>Noise Reduction]
  D --> E[Calculate RMS Current<br>& Power Values]
  E --> F[Calibration using<br>Stored Coefficients]
```

```
end
```

```
subgraph DataEncryption [Data Security Processing]
```

```
  F --> G[Encrypt Sensitive Fields<br>with AES-256-GCM]
  G --> H[Generate Message<br>Authentication Code]
  H --> I[Sign Telemetry Data<br>with ATECC608A ECDSA]
```

```
end
```

```
subgraph MQTTTransmission [MQTT Transmission]
```

```
  I --> AO[Compress Payload<br>Optional for Bandwidth]
  AO --> J[Connect to MQTT Broker<br>over TLS 1.3: Port 8883]
  J --> K[Mutual TLS Auth with<br>Device Certificate]
  K --> L[Publish to Topic:<br>smartplug/{device_id}/telemetry]
```

```
end
```

```
subgraph BackendIngestion [Backend Data Ingestion]
```

```
  L --> M[MQTT Broker Routes to<br>Backend Subscriber]
  M --> N[Backend Validates<br>Client Certificate]
  N --> O[Verify ECDSA Signature<br>with Device Public Key]
  O --> P{Signature Valid?}
```

```

    P -->|Yes| Q[Decrypt Encrypted<br>Fields with KMS]
    P -->|No| R[Log Security Event:<br>Invalid Signature]
    R --> S[Quarantine Device<br>Temporarily]
end

subgraph DataStorage [Data Storage]
    Q --> T[Store in InfluxDB<br>with Retention Policies]
    T --> U[Aggregate Data:<br>1m, 5m, 1h, 1d]
    U --> V[Update Device Metadata<br>in PostgreSQL]
end

subgraph RealTimeProcessing [Real-time Processing]
    V --> W[Check Alert Thresholds<br>for Anomaly Detection]
    W --> X{Threshold Exceeded?}
    X -->|Yes| Y[Generate Alert &<br>Trigger Notifications]
    X -->|No| Z[Continue Normal<br>Processing]

    V --> AA[WebSocket Broadcast<br>to Connected Clients]
    AA --> AB[Mobile App Updates<br>Real-time Display]
    AA --> AC[Web Dashboard Updates<br>Live Charts]
end

subgraph AnalyticsPipeline [Analytics Pipeline]
    Z --> AD[Batch Export for<br>ML Training]
    AD --> AE[Feature Engineering<br>& Pattern Recognition]
    AE --> AF[Update Anomaly<br>Detection Models]
    AF --> AG[Deploy New Models<br>to Inference Service]
    AG --> W
end

subgraph AlertProcessing [Alert Processing]
    Y --> AL[Push Notification<br>to User Mobile]
    Y --> AM[Email Alert to<br>Registered Address]
    Y --> AN[Dashboard Alert<br>for Admin View]
end

subgraph SecurityIncident [Security Incident Response]
    S --> AH[Security Review<br>Required]
    AH --> AI{Certificate<br>Revoked?}
    AI -->|Yes| AJ[Permanent Device<br>Disablement]
    AI -->|No| AK[Clear Quarantine after<br>Manual Review]
end

```

```
AK --> J
AJ --> AQ[Device Added to<br>Revocation List]
```

```
%% Styling
```

```
classDef sensor fill:#fff3e0,stroke:#ff8f00,stroke-width:2px
classDef security fill:#ffcdd2,stroke:#c62828,stroke-width:2px
classDef network fill:#bbdefb,stroke:#1565c0,stroke-width:2px
classDef backend fill:#e1bee7,stroke:#7b1fa2,stroke-width:2px
classDef storage fill:#c8e6c9,stroke:#2e7d32,stroke-width:2px
classDef realtime fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px
classDef analytics fill:#e0f2f1,stroke:#00695c,stroke-width:2px
classDef alert fill:#ffebee,stroke:#c62828,stroke-width:2px
classDef compression fill:#ffe0b2,stroke:#ef6c00,stroke-width:2px
classDef incident fill:#fce4ec,stroke:#ad1457,stroke-width:2px
```

```
class A,B,C,D,E,F sensor
class G,H,I,O,P security
class J,K,L,M,N network
class Q,R,S,T,U,V,W,X,Y backend
class T,U storage
class AA,AB,AC realtime
class AD,AE,AF,AG analytics
class Y,AL,AM,AN alert
class AO compression
class S,AH,AI,AJ,AK incident
```

```
%% Data Flow Highlight
```

```
linkStyle 0 stroke:#ff8f00,stroke-width:3px
linkStyle 4 stroke:#c62828,stroke-width:3px
linkStyle 8 stroke:#1565c0,stroke-width:3px
linkStyle 11 stroke:#7b1fa2,stroke-width:3px
linkStyle 15 stroke:#2e7d32,stroke-width:3px
linkStyle 19 stroke:#00695c,stroke-width:3px
```

```
%% Legends and Documentation
```

```
subgraph Legend [Security & Performance Features]
    direction LR
    L1[**Security:**<br>• AES-256-GCM Encryption<br>• ECDSA Signatures
(ATECC608A)<br>• Mutual TLS 1.3<br>• Certificate-based Auth]:::security
    L2[**Telemetry:**<br>• 1kHz Sampling Rate<br>• RMS Power Calculation<br>• Real-time WebSocket<br>• 1 msg/sec Rate Limit]:::sensor
    L3[**Analytics:**<br>• Anomaly Detection<br>• ML Model Training<br>• Real-time Alerts<br>• Historical Aggregation]:::analytics
end
```


end

%% Rate Limiting Note

note[**Rate Limiting:**
• MQTT: 1 message/sec per device
• WebSocket: 100 connections/device
• Database: 1000 writes/sec
• API: 100 requests/minute]:::network

6. USER AUTHENTICATION WITH 2FA ACTIVITY DIAGRAM

flowchart TD

A[User Opens Mobile App] --> B[Display Login Screen
with Email/Password]
B --> C[User Enters
Credentials]

subgraph InitialAuth [Initial Authentication]

C --> D[App Sends to Backend
over TLS 1.3]
D --> E[Backend Validates
Credentials]
E --> F{Credentials Valid?}
F -->|Yes| G[Check if 2FA
is Enabled for User]
F -->|No| H[Increment Failed
Login Counter]
H --> I{Counter > 5?}
I -->|Yes| J[Lock Account
for 15 Minutes]
I -->|No| K[Return Error:
Invalid Credentials]

end

G --> L{2FA Enabled?}

L -->|Yes| M[Generate TOTP Challenge
or Send SMS Code]
L -->|No| N[Generate JWT Token
with 30min Expiry]

subgraph TwoFactorAuth [Two-Factor Authentication]

M --> O[User Receives
2FA Code]
O --> P[User Enters
6-digit Code]
P --> Q[Backend Validates
2FA Code]
Q --> R{Code Valid?}
R -->|Yes| S[Generate JWT Token
with 2FA Flag]
R -->|No| T[Return Error:
Invalid 2FA Code]

end

subgraph SessionEstablishment [Session Establishment]

S --> U[Store Refresh Token
in Redis with TTL]
U --> V[Return Tokens to App:
• Access Token (30min)
• Refresh Token (7 days)]
N --> V

end

```

subgraph TokenManagement [Token Management]
    V --> W[App Stores Tokens<br>in Secure Storage]
    W --> X[Use Access Token for<br>Authenticated Requests]
    X --> Y{Token Expired?}
    Y -->|No| Z[Continue Normal<br>Operations]
    Y -->|Yes| AA[Use Refresh Token<br>to Get New Access Token]

    AA --> AB[Backend Validates<br>Refresh Token]
    AB --> AC{Refresh Token Valid?}
    AC -->|Yes| AD[Issue New Access Token<br>Invalidate Old]
    AC -->|No| AE[Force Re-login:<br>Tokens Revoked]
end

subgraph BiometricAuth [Optional Biometric Authentication]
    Z --> AF{Biometric Auth<br>Enabled?}
    AF -->|Yes| AG[Use FaceID/TouchID<br>for App Unlock]
    AF -->|No| AH[Continue with<br>Standard Session]
end

K --> AI[App Shows Error<br>to User]
T --> AI
J --> AJ[App Shows:<br>Account Locked]
AE --> AK[Redirect to<br>Login Screen]

%% Session Monitoring
subgraph SessionSecurity [Session Security Monitoring]
    Z --> AL[Monitor Session Activity<br>for Anomalies]
    AL --> AM{Unusual Activity<br>Detected?}
    AM -->|Yes| AN[Force Logout &<br>Require Re-auth]
    AM -->|No| AO[Continue Session<br>Normally]
end

%% Styling
classDef userAction fill:#bbdefb,stroke:#1565c0
classDef backendAction fill:#e1bee7,stroke:#7b1fa2
classDef securityCheck fill:#ffcdd2,stroke:#c62828,stroke-width:2px
classDef success fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px
classDef failure fill:#ffebee,stroke:#c62828,stroke-width:2px
classDef session fill:#fff3e0,stroke:#ff8f00
classDef biometric fill:#c8e6c9,stroke:#2e7d32

class A,B,C,P,O,AG userAction
class D,E,F,G,H,I,J,K,L,M,N,Q,R,S,T,U,V,AB,AD backendAction

```

```

class F,I,L,Q,R,Y,AC,AM securityCheck
class Z,AH,A0 success
class K,T,J,AE,AJ,AK failure
class X,AA,AL session
class AG biometric

%% Authentication Flow Highlight
linkStyle 0 stroke:#1565c0,stroke-width:2px
linkStyle 3 stroke:#c62828,stroke-width:2px
linkStyle 9 stroke:#7b1fa2,stroke-width:2px
linkStyle 14 stroke:#2e7d32,stroke-width:3px

```

```
%% Notes
```

```

note6> **Authentication Security:**<br>• TLS 1.3 for all communications<br>• Rate limiting: 5 attempts/15min<br>• JWT tokens with short expiry<br>• Refresh token rotation<br>• Biometric integration<br>• Session anomaly detection<br>• Audit logging of all auth attempts]

```

Smart Plug AI: Class & ER Diagrams

1. ENTITY-RELATIONSHIP DIAGRAM (ERD)

```
erDiagram
```

```

USERS {
    uuid user_id PK
    string email UK
    string password_hash
    string full_name
    string role
    string two_fa_secret
    boolean two_fa_enabled
    timestamp created_at
    timestamp updated_at
    timestamp last_login
    string phone_number
    boolean is_active
}

```

```

DEVICES {
    string device_id PK
    uuid owner_id FK
    string name
    string location
    string status
    boolean relay_state
}

```

```
    float current_power
    string firmware_version
    timestamp last_seen
    timestamp created_at
    timestamp updated_at
    string certificate_id
    string hardware_version
    int tamper_count
    boolean attestation_valid
    float total_energy_kwh
    int uptime_seconds
    int wifi_rssi
    string mac_address
}
```

```
POWER_READINGS {
    uuid reading_id PK
    string device_id FK
    timestamp measured_at
    float voltage
    float current
    float power
    float energy_wh
    float frequency
    float power_factor
    float temperature
    string signature
    boolean is_encrypted
}
```

```
SECURITY_EVENTS {
    uuid event_id PK
    string device_id FK
    string event_type
    string severity
    json event_data
    timestamp occurred_at
    string ip_address
    uuid user_id FK
    boolean resolved
    timestamp resolved_at
    string resolution_notes
}
```

```
COMMANDS {
    uuid command_id PK
    string device_id FK
    uuid user_id FK
    string command_type
    json command_data
    string signature
    string status
    timestamp created_at
    timestamp sent_at
    timestamp executed_at
    string result_message
    string nonce
}

CERTIFICATES {
    uuid certificate_id PK
    string device_id FK
    string cert_type
    string cert_data
    string public_key
    string private_key_slot
    date issued_date
    date expiry_date
    string status
    date revoked_date
    string revocation_reason
    string issuer_cert_id
    string serial_number
    string thumbprint
}

ALERTS {
    uuid alert_id PK
    string device_id FK
    string alert_type
    string severity
    string message
    json alert_data
    timestamp triggered_at
    timestamp acknowledged_at
    uuid acknowledged_by FK
}
```

```
    boolean resolved
    timestamp resolved_at
    string resolution_notes
    boolean notification_sent
}
```

```
AUDIT_LOGS {
    uuid log_id PK
    uuid user_id FK
    string action_type
    string resource_type
    string resource_id
    json before_state
    json after_state
    string ip_address
    string user_agent
    timestamp created_at
    string session_id
    boolean success
    string error_message
}
```

```
DEVICE_SHARING {
    uuid share_id PK
    string device_id FK
    uuid user_id FK
    uuid shared_by FK
    string access_level
    timestamp shared_at
    timestamp expires_at
    boolean is_active
    string sharing_token
}
```

```
SCHEDULES {
    uuid schedule_id PK
    string device_id FK
    uuid user_id FK
    string schedule_type
    json schedule_data
    string command
    json command_data
    boolean enabled
}
```

```

    timestamp last_executed
    timestamp next_execution
    timestamp created_at
    timestamp updated_at
}

API_KEYS {
    uuid key_id PK
    uuid user_id FK
    string api_key
    string name
    string key_type
    json allowed_ips
    json rate_limits
    timestamp created_at
    timestamp expires_at
    timestamp last_used
    int usage_count
    boolean is_active
    string revoked_reason
}

%% Relationships
USERS ||--o{ DEVICES : "owns"
USERS ||--o{ COMMANDS : "issues"
USERS ||--o{ SECURITY_EVENTS : "may trigger"
USERS ||--o{ AUDIT_LOGS : "performs"
USERS ||--o{ ALERTS : "acknowledges"
USERS ||--o{ API_KEYS : "owns"
USERS ||--o{ SCHEDULES : "creates"
USERS ||--o{ DEVICE_SHARING : "shares"

DEVICES ||--o{ POWER_READINGS : "generates"
DEVICES ||--o{ COMMANDS : "receives"
DEVICES ||--o{ SECURITY_EVENTS : "experiences"
DEVICES ||--o{ ALERTS : "triggers"
DEVICES ||--o{ CERTIFICATES : "has"
DEVICES ||--o{ DEVICE_SHARING : "shared via"
DEVICES ||--o{ SCHEDULES : "scheduled for"

CERTIFICATES ||--o{ CERTIFICATES : "issued by"

%% Many-to-Many through DEVICE_SHARING

```

```
USERS }o--o{ DEVICES : "shares access to"
```

2. CLASS DIAGRAM: CORE DOMAIN MODEL

Backend Service Layer Classes

classDiagram

```
%% Core Domain Entities
```

```
class User {
    -UUID id
    -String email
    -String passwordHash
    -String fullName
    -Role role
    -String twoFASecret
    -Boolean twoFAEnabled
    -DateTime createdAt
    -DateTime updatedAt
    -DateTime lastLogin
    -Boolean isActive
    +authenticate(password) Boolean
    +enable2FA() void
    +disable2FA() void
    +verify2FA(code) Boolean
    +hasPermission(resource, action) Boolean
    +generateJWT() String
    +validateJWT(token) Boolean
}
```

```
class Device {
    -String deviceId
    -User owner
    -String name
    -String location
    -DeviceStatus status
    -Boolean relayState
    -Float currentPower
    -String firmwareVersion
    -DateTime lastSeen
    -Certificate certificate
    -Integer tamperCount
    -Boolean attestationValid
    -Float totalEnergyKWh
    -Integer uptimeSeconds
    +turnOn() Command
}
```



```

+turnOff() Command
+toggle() Command
+getTelemetry(start, end) List~PowerReading~
+updateStatus(status) void
+verifyAttestation() Boolean
+handleTamper() void
+isOnline() Boolean
+getPowerHistory(interval) List~AggregatedReading~
}

```

```

class PowerReading {
    -UUID id
    -Device device
    -DateTime measuredAt
    -Float voltage
    -Float current
    -Float power
    -Float energyWh
    -String signature
    -Boolean isEncrypted
    +encrypt(key) void
    +decrypt(key) void
    +verifySignature() Boolean
    +calculateEnergy() Float
    +toJSON() Object
    +fromJSON(data) PowerReading
}

```

```

class SecurityEvent {
    -UUID id
    -Device device
    -SecurityEventType eventType
    -Severity severity
    -JSON eventData
    -DateTime occurredAt
    -User triggeringUser
    -Boolean resolved
    +resolve(user, notes) void
    +escalate() void
    +toAlert() Alert
    +getForensicData() JSON
    +requiresManualReview() Boolean
}

```

```

class Command {
    -UUID id
    -Device device
    -User user
    -CommandType commandType
    -JSON commandData
    -String signature
    -CommandStatus status
    -DateTime createdAt
    -String nonce
    +sign(privateKey) void
    +verifySignature(publicKey) Boolean
    +execute() Boolean
    +toMQTTPayload() String
    +fromMQTTPayload(payload) Command
    +isExpired() Boolean
}

class Certificate {
    -UUID id
    -Device device
    -CertificateType certType
    -String certData
    -String publicKey
    -String privateKeySlot
    -Date issuedDate
    -Date expiryDate
    -CertificateStatus status
    +isValid() Boolean
    +isExpired() Boolean
    +isRevoked() Boolean
    +revoke(reason) void
    +renew() Certificate
    +verifySignature(data, signature) Boolean
    +getSubject() String
    +getIssuer() String
    +getThumbprint() String
}

class Alert {
    -UUID id
    -Device device

```

```
-AlertType alertType
-Severity severity
-String message
-JSON alertData
-DateTime triggeredAt
-User acknowledgedBy
-Boolean resolved
+acknowledge(user) void
+resolve(notes) void
+escalate() void
+sendNotification() void
+toPushNotification() PushNotification
+requiresImmediateAction() Boolean
}
```

%% Enumerations

```
class Role {
    <<enumeration>>
    ADMIN
    USER
    VIEWER
    AUDITOR
    SERVICE
}
```

```
class DeviceStatus {
    <<enumeration>>
    ONLINE
    OFFLINE
    TAMPERED
    QUARANTINED
    UPDATING
    ERROR
}
```

```
class SecurityEventType {
    <<enumeration>>
    TAMPER_DETECTED
    INVALID_SIGNATURE
    ATTESTATION_FAILED
    CERTIFICATE_EXPIRED
    BRUTE_FORCE_ATTEMPT
    UNAUTHORIZED_ACCESS
}
```

```
    FIRMWARE_TAMPER
    PHYSICAL_TAMPER
}
```

```
class CommandType {
    <<enumeration>>
    RELAY_ON
    RELAY_OFF
    RELAY_TOGGLE
    RESTART
    UPDATE_INTERVAL
    OTA_UPDATE
    GET_STATUS
    UPDATE_CONFIG
}
```

```
class CertificateType {
    <<enumeration>>
    DEVICE_CERT
    SERVER_CERT
    CA_CERT
    INTERMEDIATE_CA
}
```

```
class AlertType {
    <<enumeration>>
    HIGH_POWER
    THRESHOLD_EXCEEDED
    TAMPER_ALERT
    DEVICE_OFFLINE
    ATTENTION_REQUIRED
    SECURITY_INCIDENT
    MAINTENANCE_REQUIRED
    CERTIFICATE_EXPIRY
}
```

```
%% Relationships
```

```
User "1" -- "*" Device : owns
Device "1" -- "*" PowerReading : generates
Device "1" -- "*" SecurityEvent : experiences
Device "1" -- "*" Command : receives
Device "1" -- "*" Certificate : has
Device "1" -- "*" Alert : triggers
```

```

User "1" -- "*" Command : issues
User "1" -- "*" SecurityEvent : may trigger
SecurityEvent "1" -- "1" Alert : creates
Command "1" -- "1" Certificate : signed with

%% Aggregation/Composition
Device *-- DeviceStatus
User *-- Role
SecurityEvent *-- SecurityEventType
Command *-- CommandType
Certificate *-- CertificateType
Alert *-- AlertType

%% Methods Notes
note for User "Security Methods:\n- Password hashing: BCrypt\n- 2FA:
TOTP/SMS\n- JWT: RS256 tokens\n- RBAC: Role-based access"
note for Device "Device Management:\n- Real-time status updates\n- Telemetry
collection\n- Command execution\n- Security monitoring"
note for Command "Command Security:\n- ECDSA P256 signatures\n- Nonce anti-
replay\n- Timestamp validation\n- Hardware verification via ATECC608A"
note for Certificate "Certificate Lifecycle:\n- Generation in ATECC608A\n-
Automatic renewal\n- CRL/OCSP validation\n- Key rotation quarterly"

```

3. CLASS DIAGRAM: SECURITY COMPONENTS

classDiagram

```

%% Security Service Layer
class SecurityService {
    -KeyManagementService keyService
    -CertificateService certService
    -SignatureService signatureService
    -EncryptionService encryptionService
    +authenticateDevice(certificate) Device
    +verifyCommandSignature(command, device) Boolean
    +encryptTelemetry(reading, key) EncryptedData
    +decryptTelemetry(encryptedData, key) PowerReading
    +generateChallengeResponse(device) ChallengeResponse
    +verifyAttestation(device, report) Boolean
    +handleTamperEvent(device) SecurityEvent
    +auditSecurityEvent(event) void
    +rotateKeys() void
}

class KeyManagementService {
    -Map~String, Key~ deviceKeys

```

```

-Key serverPrivateKey
-Key serverPublicKey
-Key rootCAKey
+generateDeviceKeyPair(deviceId) KeyPair
+getDevicePublicKey(deviceId) Key
+signData(data, key) Signature
+verifySignature(data, signature, key) Boolean
+encryptSymmetric(data, key) EncryptedData
+decryptSymmetric(encryptedData, key) PlainData
+rotateKey(deviceId) KeyPair
+revokeKey(deviceId) void
}

class CertificateService {
    -Certificate rootCA
    -Certificate intermediateCA
    -Map~String, Certificate~ deviceCertificates
    +generateDeviceCertificate(device, publicKey) Certificate
    +verifyCertificate(certificate) Boolean
    +checkRevocation(certificate) Boolean
    +renewCertificate(deviceId) Certificate
    +revokeCertificate(deviceId, reason) void
    +getCertificateChain(deviceId) List~Certificate~
    +validateCertificateExpiry() List~Certificate~
}

class SignatureService {
    -String algorithm "ECDSA P256"
    -String hashAlgorithm "SHA-256"
    +sign(command, privateKey) Signature
    +verify(command, signature, publicKey) Boolean
    +signTelemetry(reading, deviceKey) Signature
    +verifyTelemetry(reading, signature, deviceKey) Boolean
    +generateNonce() String
    +validateTimestamp(timestamp) Boolean
    +calculateHash(data) String
}

class EncryptionService {
    -String algorithm "AES-256-GCM"
    +encryptField(data, key) EncryptedField
    +decryptField(encryptedField, key) PlainData
    +encryptTelemetry(reading) EncryptedReading

```

```

    +decryptTelemetry(encryptedReading) PowerReading
    +generateIV() InitializationVector
    +generateAuthenticationTag(data, key) AuthTag
    +verifyAuthenticationTag(data, tag, key) Boolean
}

```

```

class ATECC608AInterface {
    -int i2cAddress
    -List~KeySlot~ keySlots
    +initialize() Boolean
    +generateKeyPair(slot) KeyPair
    +signData(data, slot) Signature
    +verifySignature(data, signature, slot) Boolean
    +readCertificate(slot) Certificate
    +writeCertificate(certificate, slot) Boolean
    +lockConfiguration() Boolean
    +getSerialNumber() String
    +getRandomNumber() String
    +incrementCounter(counterId) Integer
    +checkTamperStatus() Boolean
}

```

```

class TLSManager {
    -List~Certificate~ trustedCAs
    -Certificate serverCertificate
    -Key serverPrivateKey
    -String protocol "TLS 1.3"
    -List~String~ cipherSuites
    +createServerContext() SSLContext
    +createClientContext() SSLContext
    +verifyClientCertificate(certificate) Boolean
    +pinCertificate(certificate) void
    +checkProtocolVersion(version) Boolean
    +enablePerfectForwardSecrecy() void
    +configureSessionResumption() void
}

```

%% Hardware Security Components

```

class ESP32Security {
    -boolean secureBootEnabled
    -boolean flashEncryptionEnabled
    -String firmwareHash
    -String bootloaderHash
}

```

```

+enableSecureBoot() Boolean
+enableFlashEncryption() Boolean
+verifyFirmwareSignature() Boolean
+generateAttestationReport() AttestationReport
+checkTamperStatus() Boolean
+handleBrownout() void
+resetOnWatchdog() void
+storeSecureData(key, value) Boolean
+retrieveSecureData(key) String
}

```

```

class MAX6316Interface {
    -int watchdogTimeout
    -boolean tamperDetected
    -boolean batteryBacked
    +initialize(timeout) Boolean
    +startWatchdog() void
    +feedWatchdog() void
    +checkTamperSwitch() Boolean
    +triggerReset() void
    +enableBatteryBackup() void
    +getTamperHistory() List~TamperEvent~
    +clearTamperStatus() Boolean
}

```

```

class SecureEEPROM {
    -int i2cAddress
    -String encryptionKey
    -int storageCapacity
    +writeEncrypted(address, data) Boolean
    +readEncrypted(address) String
    +eraseSector(sector) Boolean
    +getFreeSpace() Integer
    +writeLogEntry(entry) Boolean
    +readLogEntries(count) List~LogEntry~
    +setWriteProtection(enable) Boolean
    +verifyIntegrity() Boolean
}

```

%% Data Types

```

class KeyPair {
    -Key privateKey
    -Key publicKey
}

```



```

    -String algorithm
    +getPrivateKey() Key
    +getPublicKey() Key
    +toPEM() String
    +fromPEM(pemData) KeyPair
}

class Signature {
    -String algorithm
    -byte[] signatureBytes
    -String base64Encoding
    +verify(data, publicKey) Boolean
    +toBase64() String
    +fromBase64(base64) Signature
    +getAlgorithm() String
}

class EncryptedData {
    -byte[] ciphertext
    -byte[] iv
    -byte[] authTag
    -String algorithm
    +decrypt(key) PlainData
    +getCiphertext() byte[]
    +getIV() byte[]
    +getAuthTag() byte[]
}

class AttestationReport {
    -String deviceId
    -String firmwareHash
    -boolean secureBootValid
    -boolean flashEncryptionEnabled
    -String certificateThumbprint
    -DateTime reportTime
    -String signature
    +verifySignature(publicKey) Boolean
    +toJSON() String
    +fromJSON(json) AttestationReport
    +isValid() Boolean
}

```

%% Relationships

```

SecurityService --> KeyManagementService
SecurityService --> CertificateService
SecurityService --> SignatureService
SecurityService --> EncryptionService

KeyManagementService --> KeyPair
CertificateService --> Certificate
SignatureService --> Signature
EncryptionService --> EncryptedData

ATECC608AInterface -- KeyPair : manages
ESP32Security -- AttestationReport : generates
MAX6316Interface --|> HardwareComponent

%% Dependency Injections
note for SecurityService "Central security orchestration:\n- Coordinates all
security services\n- Handles device authentication\n- Manages certificate
lifecycle\n- Processes security events"
note for ATECC608AInterface "Hardware Security Module:\n- Keys never leave
the chip\n- ECDSA P256 operations\n- 16 key slots\n- Tamper-resistant design"
note for TLSManager "Transport Layer Security:\n- TLS 1.3 exclusively\n-
Mutual TLS authentication\n- Certificate pinning\n- Perfect forward secrecy"

```

4. CLASS DIAGRAM: SERVICE LAYER

classDiagram

```

%% Service Interfaces
class DeviceService {
    -DeviceRepository deviceRepo
    -TelemetryService telemetryService
    -CommandService commandService
    -SecurityService securityService
    +registerDevice(deviceData, user) Device
    +getDevice(deviceId, user) Device
    +listDevices(user, filters) List~Device~
    +updateDevice(deviceId, updates, user) Device
    +deleteDevice(deviceId, user) Boolean
    +getDeviceTelemetry(deviceId, range, user) List~PowerReading~
    +sendCommand(deviceId, command, user) Command
    +shareDevice(deviceId, targetUser, accessLevel, user) Sharing
    +getDeviceHealth(deviceId) DeviceHealth
    +updateDeviceFirmware(deviceId, firmwareUrl, user) OTAUpdate
}

```

```

class TelemetryService {
    -InfluxDBClient influxClient
    -RedisCache redisCache
    -MQTTService mqttService
    +ingestTelemetry(deviceId, reading) Boolean
    +getRealTimeTelemetry(deviceId) PowerReading
    +getHistoricalTelemetry(deviceId, start, end, interval)
List~AggregatedReading~
    +aggregateTelemetry(deviceId, aggregation) AggregatedData
    +detectAnomalies(deviceId, reading) AnomalyResult
    +exportTelemetry(deviceId, format, user) ExportData
    +cleanupOldTelemetry(retentionDays) Integer
    +getUsageStatistics(deviceId, period) UsageStats
}

class CommandService {
    -MQTTService mqttService
    -SecurityService securityService
    -CommandRepository commandRepo
    +executeCommand(deviceId, commandType, user) Command
    +scheduleCommand(deviceId, commandType, schedule, user) ScheduledCommand
    +cancelCommand(commandId, user) Boolean
    +getCommandStatus(commandId) CommandStatus
    +listCommands(deviceId, filters, user) List~Command~
    +retryFailedCommand(commandId) Boolean
    +validateCommand(deviceId, commandType, user) ValidationResult
    +getCommandHistory(deviceId, days) List~Command~
}

class AlertService {
    -AlertRepository alertRepo
    -NotificationService notificationService
    -TelemetryService telemetryService
    +createAlert(deviceId, alertType, data) Alert
    +acknowledgeAlert(alertId, user) Alert
    +resolveAlert(alertId, user, notes) Alert
    +getActiveAlerts(user) List~Alert~
    +getAlertHistory(deviceId, filters) List~Alert~
    +configureAlertRule(deviceId, rule) AlertRule
    +testAlertRule(rule) TestResult
    +escalateAlert(alertId) Alert
    +sendAlertNotifications(alert) Boolean
}

```

```
}
```

```
class UserService {  
    -UserRepository userRepo  
    -AuthService authService  
    -RoleService roleService  
    +createUser(userData) User  
    +authenticateUser(credentials, context) AuthResult  
    +updateUser(userId, updates) User  
    +deleteUser(userId) Boolean  
    +listUsers(filters) List~User~  
    +assignRole(userId, role) User  
    +resetPassword(userId, resetData) Boolean  
    +enable2FA(userId) TwoFAConfig  
    +disable2FA(userId) Boolean  
    +getUserActivity(userId, days) List~UserActivity~  
}
```

```
class MQTTService {  
    -MqttClient mqttClient  
    -TLSManager tlsManager  
    -List~MqttSubscription~ subscriptions  
    +connectToBroker(config) Boolean  
    +publish(topic, message, qos) Boolean  
    +subscribe(topic, callback) String  
    +unsubscribe(subscriptionId) Boolean  
    +disconnect() void  
    +getConnectionStatus() ConnectionStatus  
    +handleMessage(topic, message) void  
    +reconnect() Boolean  
    +getMessageStats() MessageStatistics  
}
```

```
class NotificationService {  
    -FirebaseCloudMessaging fcm  
    -EmailService emailService  
    -SmsService smsService  
    -WebSocketService websocketService  
    +sendPushNotification(user, notification) Boolean  
    +sendEmailNotification(email, template, data) Boolean  
    +sendSMSNotification(phone, message) Boolean  
    +sendWebSocketNotification(connectionId, notification) Boolean  
    +getNotificationPreferences(user) Preferences
```

```

+updateNotificationPreferences(user, preferences) Boolean
+getNotificationHistory(user) List~Notification~
+markNotificationAsRead(notificationId) Boolean
}

```

```

class AuditService {
    -AuditRepository auditRepo
    -UserService userService
    +logEvent(user, action, resource, details) AuditLog
    +getAuditLogs(filters) List~AuditLog~
    +exportAuditLogs(format, filters) ExportData
    +searchAuditLogs(query, filters) List~AuditLog~
    +getUserAuditTrail(userId) List~AuditLog~
    +getResourceAuditTrail(resourceType, resourceId) List~AuditLog~
    +cleanupOldLogs(retentionDays) Integer
    +generateComplianceReport(period) ComplianceReport
}

```

%% Repository Interfaces

```

class DeviceRepository {
    +save(device) Device
    +findById(deviceId) Optional~Device~
    +findByOwner(userId) List~Device~
    +findByStatus(status) List~Device~
    +update(device) Device
    +delete(deviceId) Boolean
    +existsById(deviceId) Boolean
    +countByOwner(userId) Integer
    +findLastSeenBefore(timestamp) List~Device~
}

```

```

class UserRepository {
    +save(user) User
    +findById(userId) Optional~User~
    +findByEmail(email) Optional~User~
    +findAll() List~User~
    +update(user) User
    +delete(userId) Boolean
    +existsByEmail(email) Boolean
    +countByRole(role) Integer
    +findInactiveSince(timestamp) List~User~
}

```

```

class CommandRepository {
    +save(command) Command
    +findById(commandId) Optional~Command~
    +findByDevice(deviceId) List~Command~
    +findByUser(userId) List~Command~
    +findPendingCommands() List~Command~
    +update(command) Command
    +delete(commandId) Boolean
    +countByDeviceAndStatus(deviceId, status) Integer
    +findRecentCommands(limit) List~Command~
}

```

%% Data Transfer Objects

```

class DeviceDTO {
    -String deviceId
    -String name
    -String location
    -DeviceStatus status
    -Boolean relayState
    -Float currentPower
    -DateTime lastSeen
    -String firmwareVersion
    -Integer tamperCount
    +toEntity() Device
    +fromEntity(device) DeviceDTO
    +toJSON() String
}

```

```

class PowerReadingDTO {
    -String deviceId
    -DateTime measuredAt
    -Float voltage
    -Float current
    -Float power
    -Boolean isEncrypted
    +toEntity() PowerReading
    +fromEntity(reading) PowerReadingDTO
    +validate() Boolean
}

```

```

class CommandDTO {
    -String deviceId
    -CommandType commandType

```

```

    -JSON commandData
    -String nonce
    +toEntity(user) Command
    +fromEntity(command) CommandDTO
    +validate() Boolean
    +generateSignature(privateKey) String
}

```

```

class AlertDTO {
    -String deviceId
    -AlertType alertType
    -Severity severity
    -String message
    -JSON alertData
    +toEntity() Alert
    +fromEntity(alert) AlertDTO
    +shouldNotifyImmediately() Boolean
    +getNotificationMessage() String
}

```

%% Relationships

```

DeviceService --> DeviceRepository
DeviceService --> TelemetryService
DeviceService --> CommandService
DeviceService --> SecurityService

```

```

TelemetryService --> InfluxDBClient
TelemetryService --> RedisCache
TelemetryService --> MQTTService

```

```

CommandService --> MQTTService
CommandService --> SecurityService
CommandService --> CommandRepository

```

```

AlertService --> AlertRepository
AlertService --> NotificationService
AlertService --> TelemetryService

```

```

UserService --> UserRepository
UserService --> AuthService
UserService --> RoleService

```

```

NotificationService --> FirebaseCloudMessaging

```

```

NotificationService --> EmailService
NotificationService --> SmsService
NotificationService --> WebSocketService

%% Service Dependencies
note for DeviceService "Device Management Core:\n- CRUD operations\n- Telemetry access\n- Command execution\n- Sharing management\n- Firmware updates"
note for TelemetryService "Data Processing:\n- Real-time ingestion\n- Historical queries\n- Anomaly detection\n- Data aggregation\n- Export functionality"
note for CommandService "Command Orchestration:\n- Secure command execution\n- Scheduling\n- Validation\n- History tracking\n- Retry logic"
note for SecurityService "Security Orchestration:\n- Device authentication\n- Command signing\n- Telemetry encryption\n- Certificate management\n- Tamper response"

```

4. CLASS DIAGRAM: SERVICE LAYER

classDiagram

```

%% Service Interfaces
class DeviceService {
    -DeviceRepository deviceRepo
    -TelemetryService telemetryService
    -CommandService commandService
    -SecurityService securityService
    +registerDevice(deviceData, user) Device
    +getDevice(deviceId, user) Device
    +listDevices(user, filters) List~Device~
    +updateDevice(deviceId, updates, user) Device
    +deleteDevice(deviceId, user) Boolean
    +getDeviceTelemetry(deviceId, range, user) List~PowerReading~
    +sendCommand(deviceId, command, user) Command
    +shareDevice(deviceId, targetUser, accessLevel, user) Sharing
    +getDeviceHealth(deviceId) DeviceHealth
    +updateDeviceFirmware(deviceId, firmwareUrl, user) OTAUpdate
}

class TelemetryService {
    -InfluxDBClient influxClient
    -RedisCache redisCache
    -MQTTService mqttService
    +ingestTelemetry(deviceId, reading) Boolean
    +getRealTimeTelemetry(deviceId) PowerReading
    +getHistoricalTelemetry(deviceId, start, end, interval)
List~AggregatedReading~
}

```



```

+aggregateTelemetry(deviceId, aggregation) AggregatedData
+detectAnomalies(deviceId, reading) AnomalyResult
+exportTelemetry(deviceId, format, user) ExportData
+cleanupOldTelemetry(retentionDays) Integer
+getUsageStatistics(deviceId, period) UsageStats
}

class CommandService {
    -MQTTService mqttService
    -SecurityService securityService
    -CommandRepository commandRepo
    +executeCommand(deviceId, commandType, user) Command
    +scheduleCommand(deviceId, commandType, schedule, user) ScheduledCommand
    +cancelCommand(commandId, user) Boolean
    +getCommandStatus(commandId) CommandStatus
    +listCommands(deviceId, filters, user) List~Command~
    +retryFailedCommand(commandId) Boolean
    +validateCommand(deviceId, commandType, user) ValidationResult
    +getCommandHistory(deviceId, days) List~Command~
}

class AlertService {
    -AlertRepository alertRepo
    -NotificationService notificationService
    -TelemetryService telemetryService
    +createAlert(deviceId, alertType, data) Alert
    +acknowledgeAlert(alertId, user) Alert
    +resolveAlert(alertId, user, notes) Alert
    +getActiveAlerts(user) List~Alert~
    +getAlertHistory(deviceId, filters) List~Alert~
    +configureAlertRule(deviceId, rule) AlertRule
    +testAlertRule(rule) TestResult
    +escalateAlert(alertId) Alert
    +sendAlertNotifications(alert) Boolean
}

class UserService {
    -UserRepository userRepo
    -AuthService authService
    -RoleService roleService
    +createUser(userData) User
    +authenticateUser(credentials, context) AuthResult
    +updateUser(userId, updates) User

```

```

+deleteUser(userId) Boolean
+listUsers(filters) List~User~
+assignRole(userId, role) User
+resetPassword(userId, resetData) Boolean
+enable2FA(userId) TwoFAConfig
+disable2FA(userId) Boolean
+getUserActivity(userId, days) List~UserActivity~
}

```

```

class MQTTService {
    -MqttClient mqttClient
    -TLSManager tlsManager
    -List~MqttSubscription~ subscriptions
    +connectToBroker(config) Boolean
    +publish(topic, message, qos) Boolean
    +subscribe(topic, callback) String
    +unsubscribe(subscriptionId) Boolean
    +disconnect() void
    +getConnectionStatus() ConnectionStatus
    +handleMessage(topic, message) void
    +reconnect() Boolean
    +getMessageStats() MessageStatistics
}

```

```

class NotificationService {
    -FirebaseCloudMessaging fcm
    -EmailService emailService
    -SmsService smsService
    -WebSocketService websocketService
    +sendPushNotification(user, notification) Boolean
    +sendEmailNotification(email, template, data) Boolean
    +sendSMSNotification(phone, message) Boolean
    +sendWebSocketNotification(connectionId, notification) Boolean
    +getNotificationPreferences(user) Preferences
    +updateNotificationPreferences(user, preferences) Boolean
    +getNotificationHistory(user) List~Notification~
    +markNotificationAsRead(notificationId) Boolean
}

```

```

class AuditService {
    -AuditRepository auditRepo
    -UserService userService
    +logEvent(user, action, resource, details) AuditLog
}

```

```

+getAuditLogs(filters) List~AuditLog~
+exportAuditLogs(format, filters) ExportData
+searchAuditLogs(query, filters) List~AuditLog~
+getUserAuditTrail(userId) List~AuditLog~
+getResourceAuditTrail(resourceType, resourceId) List~AuditLog~
+cleanupOldLogs(retentionDays) Integer
+generateComplianceReport(period) ComplianceReport
}

```

%% Repository Interfaces

```

class DeviceRepository {
    +save(device) Device
    +findById(deviceId) Optional~Device~
    +findByOwner(userId) List~Device~
    +findByStatus(status) List~Device~
    +update(device) Device
    +delete(deviceId) Boolean
    +existsById(deviceId) Boolean
    +countByOwner(userId) Integer
    +findLastSeenBefore(timestamp) List~Device~
}

```

```

class UserRepository {
    +save(user) User
    +findById(userId) Optional~User~
    +findByEmail(email) Optional~User~
    +findAll() List~User~
    +update(user) User
    +delete(userId) Boolean
    +existsByEmail(email) Boolean
    +countByRole(role) Integer
    +findInactiveSince(timestamp) List~User~
}

```

```

class CommandRepository {
    +save(command) Command
    +findById(commandId) Optional~Command~
    +findByDevice(deviceId) List~Command~
    +findByUser(userId) List~Command~
    +findPendingCommands() List~Command~
    +update(command) Command
    +delete(commandId) Boolean
    +countByDeviceAndStatus(deviceId, status) Integer
}

```

```
    +findRecentCommands(limit) List~Command~  
}
```

%% Data Transfer Objects

```
class DeviceDTO {  
    -String deviceId  
    -String name  
    -String location  
    -DeviceStatus status  
    -Boolean relayState  
    -Float currentPower  
    -DateTime lastSeen  
    -String firmwareVersion  
    -Integer tamperCount  
    +toEntity() Device  
    +fromEntity(device) DeviceDTO  
    +toJSON() String  
}
```

```
class PowerReadingDTO {  
    -String deviceId  
    -DateTime measuredAt  
    -Float voltage  
    -Float current  
    -Float power  
    -Boolean isEncrypted  
    +toEntity() PowerReading  
    +fromEntity(reading) PowerReadingDTO  
    +validate() Boolean  
}
```

```
class CommandDTO {  
    -String deviceId  
    -CommandType commandType  
    -JSON commandData  
    -String nonce  
    +toEntity(user) Command  
    +fromEntity(command) CommandDTO  
    +validate() Boolean  
    +generateSignature(privateKey) String  
}
```

```
class AlertDTO {
```

```

    -String deviceId
    -AlertType alertType
    -Severity severity
    -String message
    -JSON alertData
    +toEntity() Alert
    +fromEntity(alert) AlertDTO
    +shouldNotifyImmediately() Boolean
    +getNotificationMessage() String
}

```

%% Relationships

```

DeviceService --> DeviceRepository
DeviceService --> TelemetryService
DeviceService --> CommandService
DeviceService --> SecurityService

```

```

TelemetryService --> InfluxDBClient
TelemetryService --> RedisCache
TelemetryService --> MQTTService

```

```

CommandService --> MQTTService
CommandService --> SecurityService
CommandService --> CommandRepository

```

```

AlertService --> AlertRepository
AlertService --> NotificationService
AlertService --> TelemetryService

```

```

UserService --> UserRepository
UserService --> AuthService
UserService --> RoleService

```

```

NotificationService --> FirebaseCloudMessaging
NotificationService --> EmailService
NotificationService --> SmsService
NotificationService --> WebSocketService

```

%% Service Dependencies

```

note for DeviceService "Device Management Core:\n- CRUD operations\n-
Telemetry access\n- Command execution\n- Sharing management\n- Firmware updates"

```

```

    note for TelemetryService "Data Processing:\n- Real-time ingestion\n- Historical queries\n- Anomaly detection\n- Data aggregation\n- Export functionality"
    note for CommandService "Command Orchestration:\n- Secure command execution\n- Scheduling\n- Validation\n- History tracking\n- Retry logic"
    note for SecurityService "Security Orchestration:\n- Device authentication\n- Command signing\n- Telemetry encryption\n- Certificate management\n- Tamper response"

```

5. ERD SUPPLEMENTAL: INDEXES & CONSTRAINTS

Optimization Indexes

```

-- Online SQL Editor to Run SQL Online.
-- Use the editor to create new tables, insert data and all other SQL operations.
-- Primary Security Indexes
CREATE INDEX idx_devices_status ON devices(status);
CREATE INDEX idx_devices_owner ON devices(owner_id);
CREATE INDEX idx_devices_last_seen ON devices(last_seen DESC);
CREATE INDEX idx_devices_tampered ON devices(tamper_count) WHERE status = 'tampered';

-- Telemetry Performance Indexes
CREATE INDEX idx_power_readings_device_time
ON power_readings(device_id, measured_at DESC);
CREATE INDEX idx_power_readings_time
ON power_readings(measured_at DESC);
CREATE INDEX idx_power_readings_encrypted
ON power_readings(device_id) WHERE is_encrypted = true;

-- Security Event Monitoring
CREATE INDEX idx_security_events_device_time
ON security_events(device_id, occurred_at DESC);
CREATE INDEX idx_security_events_severity
ON security_events(severity) WHERE resolved = false;
CREATE INDEX idx_security_events_type
ON security_events(event_type, occurred_at DESC);

-- Command History
CREATE INDEX idx_commands_device_status
ON commands(device_id, status, created_at DESC);
CREATE INDEX idx_commands_user_time
ON commands(user_id, created_at DESC);

-- Certificate Management
CREATE INDEX idx_certificates_expiry
ON certificates(expiry_date) WHERE status = 'active';
CREATE INDEX idx_certificates_device
ON certificates(device_id, status);

```

```

CREATE INDEX idx_certificates_revoked
ON certificates(revoked_date) WHERE status = 'revoked';

-- Alert System
CREATE INDEX idx_alerts_device_active
ON alerts(device_id) WHERE resolved = false;
CREATE INDEX idx_alerts_severity_time
ON alerts(severity, triggered_at DESC);
CREATE INDEX idx_alerts_notification
ON alerts(notification_sent) WHERE resolved = false;

-- Audit Logging
CREATE INDEX idx_audit_logs_user_time
ON audit_logs(user_id, created_at DESC);
CREATE INDEX idx_audit_logs_resource
ON audit_logs(resource_type, resource_id, created_at DESC);
CREATE INDEX idx_audit_logs_success
ON audit_logs(success, created_at DESC);

-- User Management
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_active ON users(is_active) WHERE is_active = true;
CREATE INDEX idx_users_last_login ON users(last_login DESC);

-- Composite Index for Common Queries
CREATE INDEX idx_device_telemetry_aggregation
ON power_readings(device_id, date_trunc('hour', measured_at), power);

```

Foreign Key Constraints

```

-- Device Ownership
ALTER TABLE devices
ADD CONSTRAINT fk_devices_owner
FOREIGN KEY (owner_id)
REFERENCES users(user_id)
ON DELETE CASCADE;

-- Device Certificates
ALTER TABLE certificates
ADD CONSTRAINT fk_certificates_device
FOREIGN KEY (device_id)
REFERENCES devices(device_id)
ON DELETE CASCADE;

-- Power Readings
ALTER TABLE power_readings
ADD CONSTRAINT fk_power_readings_device
FOREIGN KEY (device_id)
REFERENCES devices(device_id)

```

```
ON DELETE CASCADE;

-- Security Events
ALTER TABLE security_events
ADD CONSTRAINT fk_security_events_device
FOREIGN KEY (device_id)
REFERENCES devices(device_id)
ON DELETE CASCADE;

ALTER TABLE security_events
ADD CONSTRAINT fk_security_events_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE SET NULL;

-- Commands
ALTER TABLE commands
ADD CONSTRAINT fk_commands_device
FOREIGN KEY (device_id)
REFERENCES devices(device_id)
ON DELETE CASCADE;

ALTER TABLE commands
ADD CONSTRAINT fk_commands_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE SET NULL;

-- Alerts
ALTER TABLE alerts
ADD CONSTRAINT fk_alerts_device
FOREIGN KEY (device_id)
REFERENCES devices(device_id)
ON DELETE CASCADE;

ALTER TABLE alerts
ADD CONSTRAINT fk_alerts_acknowledged_by
FOREIGN KEY (acknowledged_by)
REFERENCES users(user_id)
ON DELETE SET NULL;

-- Device Sharing
ALTER TABLE device_sharing
ADD CONSTRAINT fk_device_sharing_device
FOREIGN KEY (device_id)
REFERENCES devices(device_id)
ON DELETE CASCADE;

ALTER TABLE device_sharing
```



```

ADD CONSTRAINT fk_device_sharing_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE CASCADE;

ALTER TABLE device_sharing
ADD CONSTRAINT fk_device_sharing_shared_by
FOREIGN KEY (shared_by)
REFERENCES users(user_id)
ON DELETE SET NULL;

-- Audit Logs
ALTER TABLE audit_logs
ADD CONSTRAINT fk_audit_logs_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE SET NULL;

-- API Keys
ALTER TABLE api_keys
ADD CONSTRAINT fk_api_keys_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE CASCADE;

-- Schedules
ALTER TABLE schedules
ADD CONSTRAINT fk_schedules_device
FOREIGN KEY (device_id)
REFERENCES devices(device_id)
ON DELETE CASCADE;

ALTER TABLE schedules
ADD CONSTRAINT fk_schedules_user
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE SET NULL;

```

Smart Plug AI: Interaction Sequence Diagrams

1. SECURE DEVICE PAIRING SEQUENCE

sequenceDiagram

```

title Secure Device Pairing with Challenge-Response Authentication
actor User as 👤 User
participant Mobile as 📱 Mobile App
participant Backend as ☁ Backend Server
participant DB as 🗄 Database
participant MQTT as 📡 MQTT Broker

```

participant Device as ⚡ IoT Device
participant ATECC as 📱 ATECC608A

Note over User, ATECC: Phase 1: Device Initialization

Device->>ATECC: Generate Key Pair in Slot 0

ATECC-->>Device: Private Key (never leaves chip)

Device->>Device: Store Public Key in EEPROM

Device->>Device: Generate Device ID & QR Code Data

Note over User, ATECC: Phase 2: QR Code Scanning

User->>Mobile: Open App & Navigate to Pairing

Device->>User: Display QR Code with:
• Device ID
• Public Key
•

Challenge Token

User->>Mobile: Scan QR Code

Mobile->>Mobile: Extract Device Data from QR

Mobile->>Backend: POST /api/v1/devices/pair
{deviceId, publicKey,
challenge, userId}

Note over User, ATECC: Phase 3: Challenge-Response Authentication

Backend->>DB: Store Pairing Request (pending)

Backend->>Backend: Generate Response
Signed with Server Private Key

Backend->>Mobile: Return Signed Response

Mobile->>Device: Send Response via Bluetooth/WiFi Direct

Note over User, ATECC: Phase 4: Device Verification

Device->>ATECC: Verify Server Signature
Using Server Public Key (Slot 1)

ATECC-->>Device: Signature Valid ✓

Device->>Device: Store Server Public Key
in ATECC608A Slot 1

Note over User, ATECC: Phase 5: Device Attestation

Device->>Device: Generate Attestation Report:
• Firmware Hash
• Secure
Boot Status
• Flash Encryption Status

Device->>ATECC: Sign Report with Device Private Key

Device->>MQTT: Publish Attestation over TLS 1.3

Note over User, ATECC: Phase 6: Backend Registration

MQTT->>Backend: Forward Attestation

Backend->>ATECC: Verify Attestation Signature

Backend->>Backend: Check Firmware Integrity

Backend->>DB: Create Device Record
Set status = 'pending_approval'

Backend->>Backend: Generate Device Certificate
Signed by Device CA

Backend->>MQTT: Publish Certificate to Device

MQTT->>Device: Deliver Certificate

Note over User,ATECC: Phase 7: Finalization

Device->>ATECC: Store Certificate

Device->>MQTT: Publish Registration Complete

Backend->>DB: Update device status = 'active'

Backend->>Mobile: Send Success Notification

Mobile->>User: Show "Device Paired Successfully"

Note over User,ATECC: Security Features Applied:

Note over User,ATECC: • Hardware-based key storage (ATECC608A)
•

Challenge-response anti-replay
• ECDSA P256 signatures
• Mutual authentication
• Device attestation
• TLS 1.3 encryption

2. SECURE COMMAND EXECUTION SEQUENCE

sequenceDiagram

title Secure Command Execution with ECDSA Signature Verification

actor User as  User

participant Mobile as  Mobile App

participant API as  Backend API

participant Auth as  Auth Service

participant DB as  Database

participant MQTT as  MQTT Broker

participant Device as  IoT Device

participant ATECC as  ATECC608A

participant Relay as  Relay Module

Note over User,Relay: Phase 1: User Initiation

User->>Mobile: Tap "Turn On" for Device

Mobile->>API: POST /api/v1/devices/{id}/command
Authorization: Bearer <JWT>
{command: "relay_on"}

Note over User,Relay: Phase 2: Authentication & Authorization

API->>Auth: Validate JWT Token

Auth-->>API: Token Valid (user_id, role)

API->>DB: Check RBAC Permissions
User has access to device?

DB-->>API: Permission Granted ✓

API->>DB: Rate Limiting Check
Max 10 commands/min per device

Note over User,Relay: Phase 3: Command Signing

API->>API: Generate Command Payload:
• command: "relay_on"
• timestamp: 1234567890
• nonce: "a1b2c3d4e5f6"

API->>API: Compute SHA-256 Hash of Payload
API->>API: Sign Hash with Server Private Key (ECDSA P256)
API->>DB: Log Command (pending) with Signature

Note over User,Relay: Phase 4: Secure Transmission

API->>MQTT: Publish to topic: smartplug/{device_id}/control
QoS: 1 (at least once)
Over TLS 1.3 with Client Certificate
MQTT->>Device: Deliver via mTLS (Port 8883)

Note over User,Relay: Phase 5: Device Verification

Device->>Device: Extract Signature from Payload
Device->>Device: Compute Hash of Command Data
Device->>ATECC: Verify Signature
Using Server Public Key (Slot 1)
ATECC-->>Device: Signature Valid ✓
Device->>Device: Check Timestamp Freshness (±5 min)
Device->>Device: Verify Nonce Not Replayed

Note over User,Relay: Phase 6: Command Execution

Device->>ATECC: Increment Monotonic Counter (anti-replay)
Device->>Relay: Set GPIO HIGH (3.3V)
Relay-->>Device: Relay Activated (Click)
Device->>Device: Update Internal State
Device->>Device: Store in Encrypted EEPROM

Note over User,Relay: Phase 7: Status Update

Device->>MQTT: Publish Status Update
Topic: smartplug/{device_id}/status
{status: "on", power: 60.5, ...}
MQTT->>API: Forward Status
API->>DB: Update Device State
relay_state = true, last_seen = NOW()
API->>DB: Update Command Status (executed)

Note over User,Relay: Phase 8: Real-time Feedback

API->>API: WebSocket Broadcast to Connected Clients
API->>Mobile: Push Update via WebSocket
Mobile->>User: Update UI - Show "ON" State
API->>DB: Log Audit Entry
User executed command on device

Note over User,Relay: Phase 9: Error Handling (Alternative Flow)

alt Invalid Signature
 ATECC-->>Device: Signature Invalid X
 Device->>Device: Log Security Event
 Device->>MQTT: Publish Security Alert
 MQTT->>API: Forward Alert

```
API->>DB: Mark Device as "suspicious"
API->>Mobile: Send Security Notification
end
```

```
alt Timestamp Expired
  Device->>Device: Timestamp > 5 minutes old
  Device->>Device: Discard Command (Replay Attempt)
  Device->>MQTT: Publish Replay Attempt Alert
end
```

Note over User,Relay: Security Features Applied:

Note over User,Relay: • ECDSA P256 signatures
• Nonce + timestamp anti-replay
• Hardware verification (ATECC608A)
• TLS 1.3 with mTLS
• RBAC authorization
• Rate limiting
• Complete audit trail

3. TAMPER DETECTION & RESPONSE SEQUENCE

sequenceDiagram

title Tamper Detection & Incident Response Flow

participant MAX6316 as  MAX6316 Watchdog

participant ESP32 as  ESP32-S3

participant ATECC as  ATECC608A

participant EEPROM as  24C256 EEPROM

participant MQTT as  MQTT Broker

participant Backend as  Backend Server

participant DB as  Database

participant Notify as  Notification Service

participant Mobile as  Mobile App

actor User as  User

actor Admin as  Security Admin

Note over MAX6316,Admin: Phase 1: Tamper Detection (Hardware)

MAX6316->>MAX6316: Monitor Tamper Switch (NC)
Enclosure opened → Switch opens

MAX6316->>ESP32: Trigger RESET (Active-Low)
Within 100ms of detection

ESP32->>ESP32: Immediate Hard Reset
Secure Boot verifies firmware

Note over MAX6316,Admin: Phase 2: Boot-time Tamper Processing

ESP32->>MAX6316: Check Tamper Status on Boot

MAX6316-->>ESP32: Tamper Detected ✓

ESP32->>ATECC: Check if Secure Erase Enabled

ESP32->>EEPROM: Log Tamper Event
Timestamp, counter increment

ESP32->>ESP32: Enter Safety Mode
Disable relay control

```

Note over MAX6316,Admin: Phase 3: Secure Erase (Optional)
alt Secure Erase Enabled
    ESP32->>ESP32: Erase Wi-Fi Credentials from NVS
    ESP32->>ESP32: Clear Session Tokens
    ESP32->>EEPROM: Mark for Factory Reset
else Standard Mode
    ESP32->>ESP32: Preserve credentials<br>for network alert
end

Note over MAX6316,Admin: Phase 4: Network Alert
ESP32->>ESP32: Attempt Network Connection
alt Network Available
    ESP32->>MQTT: Publish Tamper Alert<br>Over TLS 1.3 with client cert
    MQTT->>Backend: Forward Alert
else Network Unavailable
    ESP32->>EEPROM: Queue Alert for Later
    ESP32->>ESP32: Retry every 5 minutes
end

Note over MAX6316,Admin: Phase 5: Backend Processing
Backend->>DB: Log Security Event<br>Type: "physical_tamper", severity:
"critical"
Backend->>DB: Update Device Status to "tampered"
Backend->>DB: Increment tamper_count
Backend->>Backend: Check Tamper Threshold<br>If > 3 in 24h → Auto-quarantine

Note over MAX6316,Admin: Phase 6: Notification System
Backend->>Notify: Send Alert to Notification Service
Notify->>Mobile: Push Notification to User<br>"Device Tamper Detected!"
Notify->>User: Send Email Alert
Backend->>Backend: WebSocket Broadcast to Dashboard

Note over MAX6316,Admin: Phase 7: User Response
User->>Mobile: Open App, See Tamper Alert
Mobile->>User: Show Instructions:<br>1. Verify physical device<br>2. Check
tamper-evident seals<br>3. Initiate re-verification
User->>Mobile: Tap "Re-verify Device"
Mobile->>Backend: Request Re-verification

Note over MAX6316,Admin: Phase 8: Security Review
Backend->>Admin: Alert Security Dashboard

```

Admin->>Backend: Review Tamper Event
Check device history, location, patterns

alt Legitimate Tamper (Maintenance)

Admin->>Backend: Mark as "authorized_tamper"

Backend->>DB: Update event resolution

Backend->>Mobile: Send "Clear to Re-verify"

else Suspicious Activity

Admin->>Backend: Quarantine Device
Revoke certificate

Backend->>DB: Mark device as "compromised"

Backend->>User: Request Physical Inspection

end

Note over MAX6316,Admin: Phase 9: Device Recovery

Backend->>ESP32: Send Challenge via MQTT

ESP32->>ATECC: Sign Challenge Response

ESP32->>Backend: Send Signed Response

Backend->>Backend: Verify with Device Public Key

Backend->>DB: Clear Tamper Flag if valid

Backend->>ESP32: Send "Clear Tamper" Command

ESP32->>MAX6316: Reset Tamper Status

ESP32->>ESP32: Resume Normal Operations

Note over MAX6316,Admin: Phase 10: Forensic Analysis

Backend->>DB: Generate Tamper Report

DB->>Admin: Provide Forensic Data:
• Exact timestamp
• Previous device state
• Network conditions
• User location data

Admin->>DB: Add Investigation Notes

Note over MAX6316,Admin: Security Features Applied:

Note over MAX6316,Admin: • Hardware watchdog (MAX6316)
• Immediate response (100ms)
• Secure erase capability
• Multi-channel notifications
• Forensic logging
• Manual security review
• Challenge-response re-verification

4. SECURE OTA FIRMWARE UPDATE SEQUENCE

sequenceDiagram

title Secure Over-the-Air (OTA) Firmware Update Flow

participant Device as ⚡ IoT Device

participant ATECC as 🔒 ATECC608A

participant Bootloader as 📦 ESP32 Bootloader

participant MQTT as 📡 MQTT Broker

participant Backend as ☁ Backend Server

participant DB as 📄 Database

participant CDN as 🌐 CDN/Storage
participant HSM as 🏠 HSM (Signing)
participant Admin as 🔑 System Admin
participant Mobile as 📱 Mobile App

Note over Device,Mobile: Phase 1: Update Availability Check

Device->>Backend: Periodic Heartbeat
Include: firmware_version, device_id
Backend->>DB: Check for Available Updates
Compare version, compatibility
DB-->>Backend: Update Available: v1.1.0
Backend->>HSM: Generate Update Manifest
Sign with RSA-3072 Private Key
Backend->>MQTT: Publish Update Notification
{manifest_url, size, hash,
signature}

Note over Device,Mobile: Phase 2: Update Authorization

MQTT->>Device: Deliver Update Notification
Device->>ATECC: Verify Manifest Signature
Using Update CA Public Key
ATECC-->>Device: Signature Valid ✓
Device->>Device: Check Rollback Protection
New version > current version
Device->>Device: Verify Free Space (Flash)
Device->>MQTT: Publish Update Acceptance

Note over Device,Mobile: Phase 3: Firmware Download

Backend->>CDN: Prepare Firmware Binary
Encrypted with Device Public Key
CDN-->>Backend: Download URL Ready
Backend->>MQTT: Send Download Instructions
Device->>CDN: HTTPS GET firmware.bin
TLS 1.3 with certificate pinning
CDN-->>Device: Stream Encrypted Firmware
Device->>Device: Store in OTA Partition
Compute SHA-256 hash during write

Note over Device,Mobile: Phase 4: Integrity Verification

Device->>Device: Compute Final Hash
Compare with Manifest hash
alt Hashes Match
 Device->>ATECC: Verify Firmware Signature
ECDSA P256 verification
 ATECC-->>Device: Signature Valid ✓
 Device->>Device: Prepare Boot Flags
Set next boot to OTA partition
else Hash Mismatch
 Device->>Device: Delete Corrupted Firmware
 Device->>MQTT: Publish Download Failure
 Backend->>DB: Log Update Failure
end

Note over Device,Mobile: Phase 5: Update Execution


```

Device->>Device: Trigger Restart
Bootloader->>Bootloader: Check Boot Flags<br>Boot from OTA partition
Bootloader->>Bootloader: Secure Boot V2 Verification<br>RSA-PSS 3072
signature check
alt Secure Boot Passes
    Bootloader->>Device: Load New Firmware
    Device->>Device: Initialize New Version
    Device->>Device: Preserve NVS Settings
    Device->>Device: Generate Attestation Report
else Secure Boot Fails
    Bootloader->>Bootloader: Auto-rollback<br>Boot from factory partition
    Bootloader->>Device: Load Previous Firmware
    Device->>MQTT: Publish Rollback Event
end

Note over Device,Mobile: Phase 6: Post-Update Verification
Device->>ATECC: Sign Attestation Report
Device->>MQTT: Publish Attestation<br>{firmware_hash,
secure_boot_status, ...}
Backend->>Backend: Verify Attestation<br>Compare with expected values
Backend->>DB: Update Device Record<br>firmware_version = "1.1.0"
Backend->>Admin: Send Update Success Report

Note over Device,Mobile: Phase 7: Health Monitoring
Device->>Backend: Send Health Metrics<br>CPU usage, memory, stability
Backend->>Backend: Monitor for Anomalies<br>Compare with pre-update baselines
alt Anomalies Detected
    Backend->>Admin: Alert: Possible Regression
    Admin->>Backend: Schedule Rollback
else Stable Operation
    Backend->>DB: Mark Update as Stable
    Backend->>Device: Send Update Confirmation
end

Note over Device,Mobile: Phase 8: User Notification
Backend->>Mobile: Push Notification<br>"Device updated to v1.1.0"
Mobile->>Admin: Show in Admin Dashboard<br>Update status across all devices

Note over Device,Mobile: Phase 9: Batch Update Management (Alternative)
Admin->>Backend: Schedule Batch Update<br>Select devices, schedule time
Backend->>DB: Create Update Campaign
Backend->>Backend: Staged Rollout<br>10% → 50% → 100% over 24h
Backend->>Backend: Monitor Failure Rates<br>Auto-pause if > 5% failures

```

Note over Device,Mobile: Security Features Applied:

Note over Device,Mobile: • RSA-3072 signed manifests
• ECDSA firmware signatures
• TLS 1.3 for downloads
• Secure Boot V2 verification
• Rollback protection
• Device attestation
• Staged rollout with monitoring

5. REAL-TIME TELEMETRY DATA FLOW SEQUENCE

sequenceDiagram

title Real-time Telemetry Data Flow with End-to-End Encryption

participant Sensor as  SCT-013 Sensor

participant ESP32 as  ESP32-S3

participant ATECC as  ATECC608A

participant EEPROM as  24C256 EEPROM

participant MQTT as  MQTT Broker

participant Subscriber as  MQTT Subscriber

participant InfluxDB as  InfluxDB

participant PostgreSQL as  PostgreSQL

participant Redis as  Redis Cache

participant WebSocket as  WebSocket Server

participant Mobile as  Mobile App

participant Web as  Web Dashboard

participant ML as  ML Pipeline

Note over Sensor,ML: Phase 1: Sensor Data Acquisition

loop Every 1 Second

Sensor->>ESP32: Analog Signal (0-1V)
Through 33Ω burden resistor

ESP32->>ESP32: ADC Sampling (1kHz)
Digital filtering, noise reduction

ESP32->>ESP32: Calculate RMS Values
Current (A), Power (W), Voltage

(est)

ESP32->>ESP32: Apply Calibration
Using stored coefficients from EEPROM

end

Note over Sensor,ML: Phase 2: Data Encryption & Signing

ESP32->>ATECC: Generate Encryption Key
Derived from device secret

ATECC-->>ESP32: AES-256-GCM Key

ESP32->>ESP32: Encrypt Sensitive Fields
Power, current, energy (field-level)

ESP32->>ATECC: Sign Telemetry Data
ECDSA P256 with device private key

ESP32->>ESP32: Create MQTT Payload
{encrypted_data, iv, auth_tag, signature}

Note over Sensor,ML: Phase 3: Secure Transmission

```
ESP32->>MQTT: Publish to smartplug/{id}/telemetry<br>QoS: 0 (at most
once)<br>Over TLS 1.3 Port 8883
MQTT->>Subscriber: Deliver to Backend<br>ACL: Only backend can subscribe
```

```
Note over Sensor,ML: Phase 4: Backend Processing
Subscriber->>Subscriber: Verify Client Certificate<br>mTLS authentication
Subscriber->>Subscriber: Validate ECDSA Signature<br>Using device public key
Subscriber->>Subscriber: Decrypt Sensitive Fields<br>Using KMS-derived key
Subscriber->>InfluxDB: Write Time-series Data<br>With retention policies: 7d
raw, 30d 1m, etc.
Subscriber->>PostgreSQL: Update Device Metadata<br>last_seen, current_power,
status
```

```
Note over Sensor,ML: Phase 5: Real-time Distribution
Subscriber->>Redis: Cache Latest Reading<br>TTL: 5 seconds for fast access
Subscriber->>WebSocket: Broadcast to Clients<br>Authenticated WebSocket over
TLS
WebSocket->>Mobile: Push Real-time Update<br>For subscribed devices only
WebSocket->>Web: Update Dashboard<br>Live charts, gauges, metrics
```

```
Note over Sensor,ML: Phase 6: Analytics Processing
Subscriber->>Subscriber: Check Alert Thresholds<br>Configurable per device
alt Threshold Exceeded
Subscriber->>PostgreSQL: Create Alert Record
Subscriber->>Redis: Publish Alert Event
Redis->>Mobile: Push Notification
end
```

```
Subscriber->>ML: Stream for Real-time Inference<br>Anomaly detection model
ML-->>Subscriber: Anomaly Score<br>If > threshold, create security event
```

```
Note over Sensor,ML: Phase 7: Batch Processing
InfluxDB->>InfluxDB: Automatic Aggregation<br>1m, 5m, 1h, 1d aggregates
loop Every Hour
PostgreSQL->>ML: Export Batch Data<br>For model retraining
ML->>ML: Feature Engineering<br>Time-based patterns, statistics
ML->>ML: Update Anomaly Detection Model
ML->>Subscriber: Deploy New Model
end
```

```
Note over Sensor,ML: Phase 8: Data Retention & Cleanup
loop Every Day at 02:00
InfluxDB->>InfluxDB: Apply Retention Policies<br>Delete raw data > 7 days
```

```

    InfluxDB->>InfluxDB: Downsample Aggregates<br>Keep 1m for 30d, 1h for 1y
    PostgreSQL->>PostgreSQL: Archive Old Alerts<br>Move to cold storage
end

```

Note over Sensor,ML: Phase 9: Client-Side Processing

```

Mobile->>Mobile: Decrypt Display Data<br>Using app-stored keys
Mobile->>Mobile: Update UI Components<br>Real-time charts, power gauge
Mobile->>Mobile: Local Storage Cache<br>Encrypted SQLite for offline
Web->>Web: Data Visualization<br>Interactive charts, export options

```

Note over Sensor,ML: Alternative Flows

alt Network Outage

```

    ESP32->>EEPROM: Store Readings Locally<br>Encrypted with device key
    ESP32->>ESP32: Queue for Later Transmission
    ESP32->>MQTT: Burst Send when Connected<br>With sequence numbers for

```

ordering

end

alt Backend Overload

```

    Subscriber->>Redis: Rate Limit Messages<br>Drop QoS 0 if necessary
    Subscriber->>Subscriber: Scale Horizontally<br>Add more subscriber

```

instances

end

Note over Sensor,ML: Security Features Applied:

Note over Sensor,ML: • End-to-end encryption
• Field-level encryption
• ECDSA per-message signatures
• mTLS for all connections
• Rate limiting per device
• Secure data retention
• Encrypted local storage

6. USER AUTHENTICATION WITH 2FA SEQUENCE

sequenceDiagram

title User Authentication with Two-Factor Authentication (2FA)

actor User as  User

participant Browser as  Browser/App

participant LoadBalancer as  Load Balancer

participant API as  API Gateway

participant AuthService as  Auth Service

participant UserDB as  User Database

participant Redis as  Redis Cache

participant SMSService as  SMS Gateway

participant TOTPService as  TOTP Service

participant AuditLogger as  Audit Logger

participant Dashboard as  Admin Dashboard

Note over User,Dashboard: Phase 1: Initial Login Request

User->>Browser: Enter Email & Password

Browser->>LoadBalancer: HTTPS POST /auth/login
TLS 1.3, HSTS enforced

LoadBalancer->>API: Route to Auth Endpoint
With rate limiting (5/min per IP)

API->>AuthService: Forward Credentials

Note over User,Dashboard: Phase 2: Credential Validation

AuthService->>UserDB: Lookup User by Email

UserDB-->>AuthService: User Record + Password Hash

AuthService->>AuthService: Verify BCrypt Hash

alt Invalid Credentials

AuthService->>Redis: Increment Failed Attempts

AuthService->>AuditLogger: Log Failed Attempt
IP, user_agent, timestamp

AuthService->>API: Return 401 Unauthorized

API->>Browser: Show Error Message

Note over AuthService: Account lock after 5 failures (15min)

end

Note over User,Dashboard: Phase 3: 2FA Requirement Check

AuthService->>UserDB: Check 2FA Configuration

UserDB-->>AuthService: 2FA enabled: true, method: "TOTP"

AuthService->>API: Return 202 Accepted
Requires 2FA token

API->>Browser: Show 2FA Input Screen

Note over User,Dashboard: Phase 4: TOTP Verification

User->>Browser: Enter 6-digit TOTP Code
From Google Authenticator/Authy

Browser->>API: POST /auth/verify-2fa
{email, totp_code}

API->>AuthService: Forward 2FA Request

AuthService->>TOTPService: Verify TOTP Code
Using user's secret from DB

TOTPService-->>AuthService: Code Valid ✓

Note over User,Dashboard: Alternative: SMS 2FA

alt SMS 2FA Enabled

AuthService->>SMSService: Send SMS Code
To user's verified phone

SMSService->>User: Deliver SMS
"Your code: 123456"

User->>Browser: Enter SMS Code

end

Note over User,Dashboard: Phase 5: Session Creation

AuthService->>AuthService: Generate JWT Access Token
Expires: 30 minutes,
algorithm: RS256

AuthService->>AuthService: Generate Refresh Token
Expires: 7 days, stored
in Redis

AuthService->>Redis: Store Session Data
{user_id, roles, permissions, ip}

AuthService->>UserDB: Update last_login timestamp

Note over User,Dashboard: Phase 6: Response Delivery

AuthService->>API: Return Auth Tokens
{access_token, refresh_token,
expires_in}

API->>Browser: Set Secure Cookies
HttpOnly, Secure, SameSite=Strict

Browser->>Browser: Store Tokens in Memory
NOT localStorage (XSS
protection)

Note over User,Dashboard: Phase 7: Subsequent Requests

Browser->>API: Request with Authorization Header
Bearer <access_token>

API->>AuthService: Validate JWT Signature

AuthService->>Redis: Check Token Blacklist

AuthService->>API: Return User Context
{user_id, roles, permissions}

API->>Dashboard: Process Request with RBAC

Note over User,Dashboard: Phase 8: Token Refresh Flow

Browser->>API: POST /auth/refresh
{refresh_token}

API->>AuthService: Validate Refresh Token

AuthService->>Redis: Verify Token Not Revoked

AuthService->>AuthService: Issue New Access Token
Invalidate old token

AuthService->>API: Return New Tokens

API->>Browser: Update Client Tokens

Note over User,Dashboard: Phase 9: Security Monitoring

AuthService->>AuditLogger: Log Successful Login
Timestamp, IP, device,
location

AuditLogger->>Dashboard: Real-time Security Dashboard
Show active
sessions, failed attempts

loop Every 5 Minutes

AuthService->>Redis: Clean Expired Sessions

AuthService->>UserDB: Update Inactive Users

end

Note over User,Dashboard: Phase 10: Logout/Invalidation

User->>Browser: Click "Logout"

Browser->>API: POST /auth/logout

API->>AuthService: Invalidate Tokens

AuthService->>Redis: Add to Blacklist
Access token & refresh token
AuthService->>AuditLogger: Log Logout Event
API->>Browser: Clear Auth Cookies

Note over User,Dashboard: Alternative Flows

alt Biometric Authentication

Browser->>Browser: Request FaceID/TouchID

Browser->>API: Send Biometric Proof

API->>AuthService: Verify Biometric Signature

AuthService->>API: Skip 2FA if biometric valid

end

alt Device Recognition

AuthService->>AuthService: Check Device Fingerprint
Browser, OS,
location patterns

AuthService->>API: Require 2FA for new devices

AuthService->>API: Skip 2FA for trusted devices

end

Note over User,Dashboard: Security Features Applied:

Note over User,Dashboard: • TLS 1.3 for all connections
• BCrypt password
hashing
• JWT with RS256 signatures
• HttpOnly Secure cookies
• Rate
limiting per IP/user
• Comprehensive audit logging
• Device
fingerprinting
• Token blacklisting
• Biometric integration

Smart Plug AI: Gantt Chart & Project Timeline

1.DETAILED PHASE 1 GANTT CHART (Weeks 1-12)

gantt

title Phase 1: Secure MVP Development (Detailed Week-by-Week)

dateFormat YYYY-MM-DD

axisFormat %W

section Week 1-2: Hardware & Firmware Foundation

Component Testing & Verification :w1_1, 2025-01-06, 4d

ATECC608A Provisioning :w1_2, 2025-01-08, 3d

Secure Boot + Flash Encryption :w1_3, 2025-01-11, 2d

MQTT over TLS 1.3 Setup :w1_4, 2025-01-13, 3d

Tamper Detection Integration :w1_5, 2025-01-16, 2d

Relay Control System :w1_6, 2025-01-18, 2d

Power Measurement Calibration :w1_7, 2025-01-20, 2d

section Week 3-4: Mobile App Security Features

Flutter App Setup with 2FA :w3_1, 2025-01-20, 3d

Real-time Power Monitoring UI	:w3_2, 2025-01-23, 3d
Secure QR Code Pairing	:w3_3, 2025-01-26, 4d
Device Control with Signatures	:w3_4, 2025-01-30, 2d
Historical Data Display	:w3_5, 2025-02-01, 2d
Security Status Dashboard	:w3_6, 2025-02-03, 2d
Push Notifications System	:w3_7, 2025-02-05, 2d

section Week 5-6: Cloud Backend Security

Firestore with Field Encryption	:w5_1, 2025-02-05, 3d
Cloud Functions with RBAC	:w5_2, 2025-02-08, 3d
WebSocket over TLS	:w5_3, 2025-02-11, 3d
MQTT Broker with mTLS	:w5_4, 2025-02-14, 3d
Signed Command System	:w5_5, 2025-02-17, 3d
Device Attestation Endpoints	:w5_6, 2025-02-20, 2d
Audit Logging System	:w5_7, 2025-02-22, 2d

section Week 7-8: Safety & Enclosure

PCB Design with Security Zones	:w7_1, 2025-02-24, 4d
3D-printed Enclosure v1	:w7_2, 2025-02-28, 3d
Mains Wiring with Safety	:w7_3, 2025-03-03, 4d
Thermal Monitoring System	:w7_4, 2025-03-07, 2d
Overload Protection Circuit	:w7_5, 2025-03-09, 2d
Physical Tamper Mechanisms	:w7_6, 2025-03-11, 2d
Assembly Documentation	:w7_7, 2025-03-13, 3d

section Week 9-10: Rule Engine & Automation

Time-based Scheduling System	:w9_1, 2025-03-17, 3d
Manual Rules with 2FA	:w9_2, 2025-03-20, 3d
Encrypted Alert System	:w9_3, 2025-03-23, 2d
Load-shedding Detection	:w9_4, 2025-03-25, 2d
Rate Limiting Implementation	:w9_5, 2025-03-27, 2d
Anti-replay Protection	:w9_6, 2025-03-29, 3d
Rule Engine Testing	:w9_7, 2025-04-01, 3d

section Week 11-12: Testing & Security Audit

10-device Pilot Deployment	:w11_1, 2025-04-03, 3d
User Testing & Feedback	:w11_2, 2025-04-06, 5d
Security Penetration Testing	:w11_3, 2025-04-11, 4d
Bug Fixing & Improvements	:w11_4, 2025-04-15, 3d
UI/UX Refinement	:w11_5, 2025-04-18, 3d
ML Data Collection Setup	:w11_6, 2025-04-21, 2d
Security Audit Report	:w11_7, 2025-04-23, 2d

%% Milestones

Hardware Ready :milestone_hw, 2025-01-20, 0d
Mobile App Ready :milestone_app, 2025-02-05, 0d
Backend Ready :milestone_backend, 2025-02-24, 0d
Enclosure Ready :milestone_enclosure, 2025-03-17, 0d
Phase 1 Complete :milestone_phase1, 2025-04-25, 0d

2. SECURITY MILESTONE TIMELINE

gantt

title Security Milestone Timeline
dateFormat YYYY-MM-DD
axisFormat %Y-%m

section HARDWARE SECURITY

ATECC608A Provisioning Complete :hw_1, 2025-01-13, 0d
Secure Boot V2 Enabled :hw_2, 2025-01-15, 0d
Flash Encryption Active :hw_3, 2025-01-17, 0d
Tamper Detection Operational :hw_4, 2025-01-20, 0d
Encrypted EEPROM Functional :hw_5, 2025-01-22, 0d

section COMMUNICATION SECURITY

MQTT over TLS 1.3 Operational :comms_1, 2025-01-25, 0d
Client Certificate Authentication :comms_2, 2025-01-28, 0d
Signed Commands Implementation :comms_3, 2025-02-05, 0d
WebSocket over TLS :comms_4, 2025-02-12, 0d
Certificate Pinning :comms_5, 2025-02-15, 0d

section APPLICATION SECURITY

2FA Authentication :app_1, 2025-02-08, 0d
RBAC Implementation :app_2, 2025-02-15, 0d
Input Validation Complete :app_3, 2025-02-20, 0d
Audit Logging System :app_4, 2025-02-25, 0d
Rate Limiting Active :app_5, 2025-03-05, 0d

section DATA SECURITY

Field-level Encryption :data_1, 2025-03-10, 0d
End-to-end Encryption :data_2, 2025-03-15, 0d
Secure Key Management :data_3, 2025-03-20, 0d
Data Retention Policies :data_4, 2025-03-25, 0d
Backup Encryption :data_5, 2025-03-30, 0d

section COMPLIANCE & CERTIFICATION

Initial Security Audit :comp_1, 2025-04-15, 0d

SABS Compliance Testing :comp_2, 2025-06-01, 0d
ICASA Type Approval :comp_3, 2025-07-15, 0d
SOC2 Type I Controls :comp_4, 2025-09-01, 0d
GDPR Readiness :comp_5, 2025-11-01, 0d

section CONTINUOUS SECURITY

Bug Bounty Program Launch :cont_1, 2025-06-15, 0d
24/7 Security Monitoring :cont_2, 2025-08-01, 0d
Automated Security Scanning :cont_3, 2025-09-15, 0d
Incident Response Team :cont_4, 2025-10-01, 0d
Quarterly Penetration Tests :cont_5, 2025-12-01, 0d

3. RISK MANAGEMENT TIMELINE

gantt

title Risk Management & Mitigation Timeline
dateFormat YYYY-MM
axisFormat %Y-%m

section TECHNICAL RISKS

Hardware Security Implementation :risk_tech_1, 2025-01, 1M
Firmware Stability & Reliability :risk_tech_2, 2025-02, 1M
Scalability Testing :risk_tech_3, 2025-03, 1M
Integration Testing :risk_tech_4, 2025-04, 1M
Performance Optimization :risk_tech_5, 2025-05, 1M

section SECURITY RISKS

Vulnerability Assessment :risk_sec_1, 2025-02, 1M
Penetration Testing Phase 1 :risk_sec_2, 2025-04, 1M
Security Audit & Remediation :risk_sec_3, 2025-06, 1M
Penetration Testing Phase 2 :risk_sec_4, 2025-08, 1M
Continuous Threat Monitoring :risk_sec_5, 2025-10, 2M

section REGULATORY RISKS

SABS Compliance Preparation :risk_reg_1, 2025-03, 2M
ICASA Certification Process :risk_reg_2, 2025-05, 3M
POPIA/GDPR Compliance :risk_reg_3, 2025-07, 2M
SOC2 Type I Preparation :risk_reg_4, 2025-09, 2M
International Compliance :risk_reg_5, 2025-11, 2M

section MARKET RISKS

Market Research Validation :risk_mkt_1, 2025-01, 2M
Competitive Analysis :risk_mkt_2, 2025-03, 2M
Pricing Strategy Testing :risk_mkt_3, 2025-05, 2M

Customer Feedback Integration :risk_mkt_4, 2025-07, 2M
Market Expansion Planning :risk_mkt_5, 2025-09, 3M

section OPERATIONAL RISKS

Supply Chain Setup :risk_ops_1, 2025-02, 3M
Manufacturing Process :risk_ops_2, 2025-05, 3M
Distribution Logistics :risk_ops_3, 2025-08, 2M
Customer Support Setup :risk_ops_4, 2025-10, 2M
Disaster Recovery Planning :risk_ops_5, 2025-12, 2M

4. DELIVERABLE TIMELINE

gantt

title Key Deliverables Timeline
dateFormat YYYY-MM-DD
axisFormat %Y-%m

section PROTOTYPE DELIVERABLES (Phase 1)

Working Breadboard Prototypes (3 units) :deliverable_1, 2025-01-20, 0d
Mobile App v0.1 with Security Features :deliverable_2, 2025-02-05, 0d
Secure Cloud Backend v1 :deliverable_3, 2025-02-24, 0d
Enclosed Secure Prototypes (3 units) :deliverable_4, 2025-03-17, 0d
Secure Rule Engine v1 :deliverable_5, 2025-04-01, 0d
Security Audit Report v1 :deliverable_6, 2025-04-25, 0d

section PILOT DELIVERABLES (Phase 2)

Production-ready PCB Design :deliverable_7, 2025-05-15, 0d
Software v2.0 with Analytics :deliverable_8, 2025-06-30, 0d
500-device Pilot Deployment :deliverable_9, 2025-07-31, 0d
SABS/ICASA Certification :deliverable_10, 2025-08-31, 0d
Manufacturing Process Documentation :deliverable_11, 2025-09-15, 0d

section COMMERCIAL DELIVERABLES (Phase 3)

Retail Launch Event :deliverable_12, 2025-10-15, 0d
First 10,000 Units Shipped :deliverable_13, 2025-11-30, 0d
Enterprise Features Complete :deliverable_14, 2025-12-15, 0d
SOC2 Type I Certification :deliverable_15, 2026-01-31, 0d
50,000-device Capacity :deliverable_16, 2026-02-28, 0d

section DOCUMENTATION DELIVERABLES

Technical Specifications v1.0 :doc_1, 2025-02-15, 0d
Security Architecture Document :doc_2, 2025-03-15, 0d
User Manual & Installation Guide :doc_3, 2025-05-15, 0d
API Documentation v1.0 :doc_4, 2025-07-15, 0d

Compliance Documentation :doc_5, 2025-09-15, 0d
Operations Manual :doc_6, 2025-11-15, 0d